

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №8
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Обработка аудиосигналов с помощью нейронных сетей.

Цель работы: изучение методов автоматической классификации аудиоданных с применением нейронных сетей различных архитектур.

Репозиторий: https://github.com/bickrosss/NN_lab8

Порядок выполнения работы:

1.1. Реализация на Keras (полносвязная сеть).

1.1.1. Импорт библиотек.

Выполняется импорт NumPy, Matplotlib, gdown, os, time, random, math, pickle, librosa. Из TensorFlow/Keras подключаются: to_categorical, оптимизаторы Adam и RMSprop, Sequential, Model, слои Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, Conv2D, LSTM. Из sklearn – train_test_split, LabelEncoder, StandardScaler, confusion_matrix и ConfusionMatrixDisplay.

```

# Массивы
import numpy as np

# Отрисовка графиков
import matplotlib.pyplot as plt

# Загрузка из google облака
import gdown

# Преобразование категориальных данных в one hot encoding
from tensorflow.keras.utils import to_categorical

# Работа с папками и файлами
import os

# Утилиты работы со временем
import time

# Работа со случайными числами
import random

# Математические функции
import math

# Сохранение и загрузка структур данных Python
import pickle

# Параметризация аудио
import librosa

# Оптимизаторы для обучения моделей
from tensorflow.keras.optimizers import Adam, RMSprop

# Конструирование и загрузка моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model, load_model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, Conv2D, LSTM

# Разбиение на обучающую и проверочную выборку
from sklearn.model_selection import train_test_split

# Кодирование категориальных меток, нормирование числовых данных
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline

```

Рисунок 1.1 – Импорт библиотек

1.1.2. Загрузка датасета и установка констант.

Датасет GTZAN загружается из облачного хранилища (архив genres.zip, 1.23 ГБ). Дополнительно загружаются предподготовленные данные audio_data_mean.pickle. После распаковки проверяется структура: 10 жанровых классов (blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, rock), по 100 файлов формата .au. Устанавливаются константы: CLASS_FILES=100, FILE_INDEX_TRAIN_SPLIT=90, VALIDATION_SPLIT=0.1, DURATION_SEC=30, N_FFT=8192, HOP_LENGTH=512.

✓ Загрузка датасета и подготовка данных

```
2) S3
CBL # Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/genres.zip', None, quiet=False)

# Загрузка подготовленных данных
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/audio_data_mean.pickle', None, quiet=True)

✓ Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/112/genres.zip
To: /content/genres.zip
100% [██████████] 1.23G/1.23G [00:51<00:00, 23.7MB/s]
'audio_data_mean.pickle'

1) 14
CBL # Распаковка архива на локальный диск colab
!unzip -qo genres.zip

# Проверка выгруженных папок
!ls genres

# Проверка содержимого одной папки
!ls genres/blues

✓ blues classical country disco hip-hop jazz metal pop reggae rock
blues.00000.au blues.00020.au blues.00040.au blues.00060.au blues.00080.au
blues.00001.au blues.00021.au blues.00041.au blues.00061.au blues.00081.au
blues.00002.au blues.00022.au blues.00042.au blues.00062.au blues.00082.au
blues.00003.au blues.00023.au blues.00043.au blues.00063.au blues.00083.au
blues.00004.au blues.00024.au blues.00044.au blues.00064.au blues.00084.au
blues.00005.au blues.00025.au blues.00045.au blues.00065.au blues.00085.au
blues.00006.au blues.00026.au blues.00046.au blues.00066.au blues.00086.au
blues.00007.au blues.00027.au blues.00047.au blues.00067.au blues.00087.au
blues.00008.au blues.00028.au blues.00048.au blues.00068.au blues.00088.au
blues.00009.au blues.00029.au blues.00049.au blues.00069.au blues.00089.au
blues.00010.au blues.00030.au blues.00050.au blues.00070.au blues.00090.au
blues.00011.au blues.00031.au blues.00051.au blues.00071.au blues.00091.au
blues.00012.au blues.00032.au blues.00052.au blues.00072.au blues.00092.au
blues.00013.au blues.00033.au blues.00053.au blues.00073.au blues.00093.au
blues.00014.au blues.00034.au blues.00054.au blues.00074.au blues.00094.au
blues.00015.au blues.00035.au blues.00055.au blues.00075.au blues.00095.au
blues.00016.au blues.00036.au blues.00056.au blues.00076.au blues.00096.au
blues.00017.au blues.00037.au blues.00057.au blues.00077.au blues.00097.au
blues.00018.au blues.00038.au blues.00058.au blues.00078.au blues.00098.au
blues.00019.au blues.00039.au blues.00059.au blues.00079.au blues.00099.au

6)
✓ # Установка констант

FILE_DIR = './genres' # Папка с файлами датасета
CLASS_LIST = os.listdir(FILE_DIR) # Список классов, порядок меток не определен!
CLASS_LIST.sort() # Сортировка списка классов для фиксации порядка меток
CLASS_COUNT = len(CLASS_LIST) # Количество классов
CLASS_FILES = 100 # Общее количество файлов в каждом классе
FILE_INDEX_TRAIN_SPLIT = 90 # Количество файлов каждого класса на основной набор
VALIDATION_SPLIT = 0.1 # Доля проверочной выборки в основном наборе
DURATION_SEC = 30 # Анализируемая длительность аудиосигнала
N_FFT = 8192 # Размер окна преобразования Фурье для расчета спектра
HOP_LENGTH = 512 # Объем данных для расчета одного набора признаков

5) 0
CBL # Проверка списка классов
print(CLASS_LIST)

✓ ['blues', 'classical', 'country', 'disco', 'hip-hop', 'jazz', 'metal', 'pop', 'reggae', 'rock']
```

Рисунок 1.2 – Загрузка датасета и установка констант

1.1.3. Функции параметризации и извлечения признаков.

Функция `get_features` вычисляет усреднённые акустические признаки: хромограмма, MFCC (20 коэф.), RMS, спектральный центроид, ширина полосы частот, спектральный rolloff, пересечения нуля. Функция `stack_features` объединяет их в единый вектор. Функция `get_feature_list_from_file` загружает аудиофайл и возвращает матрицу признаков с one-hot меткой.

<pre> # Функция параметризации аудио def get_features(y, # волновое представление сигнала sr, # частота дискретизации сигнала y n_fft=N_FFT, # размер скользящего окна БПФ hop_length=HOP_LENGTH # шаг скользящего окна БПФ): # Вычисление различных параметров (признаков) аудио # Хромограмма chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length) # Мел-кепстральные коэффициенты mfcc = librosa.feature.mfcc(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length) # Среднеквадратическая амплитуда rmse = librosa.feature.rms(y=y, hop_length=hop_length) # Спектральный центроид spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length) # Ширина полосы частот spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length) # Спектральный спад частоты rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length) # Пересечения нуля zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length) # Сборка параметров в общий список: # На один файл один усредненный вектор признаков features = {'rmse': rmse.mean(axis=1, keepdims=True), 'spct': spec_cent.mean(axis=1, keepdims=True), 'spbw': spec_bw.mean(axis=1, keepdims=True), 'roff': rolloff.mean(axis=1, keepdims=True), 'zcr': zcr.mean(axis=1, keepdims=True), 'mfcc': mfcc.mean(axis=1, keepdims=True), 'stft': chroma_stft.mean(axis=1, keepdims=True)} return features </pre>	
<pre> # Функция объединения признаков в набор векторов def stack_features(feats # словарь признаков, отдельные векторы по ключу каждого признака): features = None for v in feats.values(): features = np.vstack((features, v)) if features is not None else v return features.T </pre>	
<pre> # Функция формирования набора признаков и метки класса для аудиофайла def get_feature_list_from_file(class_index, # индекс класса файла song_name song_name, # имя аудиофайла duration_sec # длительность аудио в секундах): # Загрузка в y первых duration_sec секунд аудиосигнала y, sr = librosa.load(song_name, mono=True, duration=duration_sec) # Извлечение параметров из аудиосигнала features = get_features(y, sr) feature_set = stack_features(features) # Перевод номера класса в one hot encoding y_label = to_categorical(class_index, CLASS_COUNT) return feature_set, y_label </pre>	

Рисунок 1.3 – Функции параметризации аудио и сборки признаков

1.1.4. Функции формирования датасета.

Функция `process_file` обрабатывает один аудиофайл заданного класса. Функция `extract_data` обходит все классы и файлы в диапазоне индексов, собирая итоговые массивы `x_data` (N, 37) и `y_data` (N, 10) с помощью `np.vstack`.

```

# Функция формирования подвыборки признаков и меток класса для одного файла

def process_file(class_index, # индекс класса аудиофайла
                file_index,  # индекс (порядковый номер) аудиофайла в папке класса
                duration_sec  # длительность аудио в секундах
                ):
    x_list = []
    y_list = []
    class_name = CLASS_LIST[class_index]

    # Извлечение имени произведения
    song_name = f'{FILE_DIR}/{class_name}/{class_name}.{str(file_index).zfill(5)}.au'

    # Выборка признаков и метки класса для произведения
    feature_set, y_label = get_feature_list_from_file(class_index,
                                                    song_name,
                                                    duration_sec)

    # Добавление данных в наборы
    for j in range(feature_set.shape[0]):
        x_list.append(feature_set[j])
        y_list.append(y_label)

    # Возврат имени файла и numpy-массивов признаков и меток класса
    return song_name, \
        np.array(x_list).astype('float32'), \
        np.array(y_list).astype('float32')

# Функция формирования набора данных из файлов всех классов по диапазону номеров файлов

def extract_data(file_index_start, # начальный индекс аудиофайла
                file_index_end,   # конечный индекс аудиофайла (не достигая)
                duration_sec=DURATION_SEC # длительность аудио в секундах
                ):
    # Списки для последовательностей входных данных и меток класса
    x_data = None
    y_data = None

    # Фиксация времени старта формирования выборки
    curr_time = time.time()

    # Для всех классов:
    for class_index in range(len(CLASS_LIST)):
        # Для всех файлов текущего класса из заданного диапазона номеров:
        for file_index in range(file_index_start, file_index_end):
            # Обработка одного файла и добавление данных к общим массивам
            _, file_x_data, file_y_data = process_file(class_index, file_index, duration_sec)
            x_data = file_x_data if x_data is None else np.vstack([x_data, file_x_data])
            y_data = file_y_data if y_data is None else np.vstack([y_data, file_y_data])

        # Вывод информации о готовности обработки датасета
        print(f'Жанр {CLASS_LIST[class_index]} готов -> {round(time.time() - curr_time)} c')
        curr_time = time.time()

    # Возврат массивов набора данных
    return x_data, y_data

```

Рисунок 1.4 – Функции process_file и extract_data

1.1.5. Извлечение признаков, нормировка и разбиение.

Вызов extract_data для файлов 0–89 даёт x_train_data формы (900, 37). Нормировка: StandardScaler. Разбиение train_test_split со stratify: x_train (810, 37), x_val (90, 37), y_train (810, 10), y_val (90, 10).

<pre> # Извлекаем признаки из аудиофайлов ТЕМ ЖЕ способом, что используется в classify_test_files. x_train_data, y_train_data = extract_data(file_index_start=0, file_index_end=FILE_INDEX_TRAIN_SPLIT, duration_sec=DURATION_SEC) print('Форма x_train_data:', x_train_data.shape) print('Форма y_train_data:', y_train_data.shape) </pre>	
<pre> --- Жанр blues готов -> 126 с Жанр classical готов -> 111 с Жанр country готов -> 111 с Жанр disco готов -> 111 с Жанр hip-hop готов -> 111 с Жанр jazz готов -> 112 с Жанр metal готов -> 112 с Жанр pop готов -> 111 с Жанр reggae готов -> 111 с Жанр rock готов -> 110 с Форма x_train_data: (900, 37) Форма y_train_data: (900, 10) </pre>	
<pre> # Нормирование признаков в соответствии со стандартным нормальным распределением x_scaler = StandardScaler() x_train_data_scaled = x_scaler.fit_transform(x_train_data) </pre>	
<pre> # Разделение набора данных на обучающую и проверочную выборки # Параметр stratify указывает метки классов, по которым происходит балансировка разделения x_train, x_val, y_train, y_val = train_test_split(x_train_data_scaled, y_train_data, stratify=y_train_data, test_size=VALIDATION_SPLIT) </pre>	

Рисунок 1.5 – Извлечение признаков, нормировка и разбиение

1.1.6. Вспомогательные функции визуализации.

Функция `show_history` строит графики `accuracy/val_accuracy` и `loss/val_loss` по эпохам. Функция `classify_file` классифицирует один файл: извлекает признаки, нормирует, получает предсказание голосованием, визуализирует результат столбчатой диаграммой.

Функция вывода графиков точности и ошибки по эпохам обучения

```
# Вывод графиков точности и ошибки распознавания на обучающей и проверочной выборках

def show_history(history # объект-результат метода обучения .fit()
):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle('График процесса обучения модели')
    ax1.plot(history.history['accuracy'],
              label='Доля верных ответов на обучающем наборе')
    ax1.plot(history.history['val_accuracy'],
              label='Доля верных ответов на проверочном наборе')
    ax1.xaxis.get_major_locator().set_params(integer=True)
    ax1.set_xlabel('Эпоха обучения')
    ax1.set_ylabel('Доля верных ответов')
    ax1.legend()

    ax2.plot(history.history['loss'],
              label='Ошибка на обучающем наборе')
    ax2.plot(history.history['val_loss'],
              label='Ошибка на проверочном наборе')
    ax2.xaxis.get_major_locator().set_params(integer=True)
    ax2.set_xlabel('Эпоха обучения')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()
```

Функции визуализации распознавания отдельных звуковых файлов

```
# Классификация файла и визуализация предсказания модели для него

def classify_file(model, # обученная модель классификатора
                  x_scaler, # настроенный нормировщик входных данных
                  class_index, # верный индекс класса аудиофайла
                  file_index # индекс (порядковый номер) аудиофайла в папке
                  ):
    # Подготовка выборки данных файла произведений
    song_name, file_x_data, file_y_data = process_file(class_index, file_index, DURATION_SEC)

    # Нормирование признаков уже настроенным нормировщиком
    file_x_data = x_scaler.transform(file_x_data)

    print('Файл:', song_name)
    print('Векторы для предсказания:', file_x_data.shape)

    # Вычисление предсказания по выборке
    predict = model.predict(file_x_data)
    # Определение среднего предсказания (голосование)
    predict_mean = predict.mean(axis=0)
    # Определение индекса класса по результату голосования
    predict_class_index = np.argmax(predict_mean)
    # Вычисление признака правильного предсказания
    predict_good = predict_class_index == class_index

    # Визуализация предсказания сети для файла
    plt.figure(figsize=(10,3))
    print('Классификация сети:', CLASS_LIST[predict_class_index], '-', 'ВЕРНО :-)' if predict_good else 'НЕВЕРНО.')
    plt.title('Среднее распределение векторов предсказаний')
    plt.bar(CLASS_LIST, predict_mean, color='g' if predict_good else 'r')
    plt.show()
    print('-----')

    # Возврат результата предсказания
    return predict_class_index
```

Рисунок 1.6 – Функции show_history и classify_file

1.1.7. Функция пакетной классификации тестовых файлов.

Функция `classify_test_files` принимает модель, нормировщик, начальный индекс и количество файлов на класс. Накапливает `y_true` и `y_pred`, вычисляет долю верных предсказаний и строит матрицу ошибок.


```

> # Классификация и визуализация нескольких файлов каждого класса

def classify_test_files(model,          # обученная модель классификатора
                      x_scaler,       # настроенный нормировщик входных данных
                      from_index,     # индекс аудиофайла, с которого начинать визуализацию
                      n_files):       # количество файлов для визуализации

    predict_all = 0
    predict_good = 0
    y_true = []
    y_pred = []

    # Классификация каждого файла и аккумуляция результатов классификации
    for class_index in range(CLASS_COUNT):
        for file_index in range(from_index, from_index + n_files):
            predict_class_index = classify_file(model, x_scaler, class_index, file_index)
            y_true.append(class_index)
            y_pred.append(predict_class_index)
            predict_all += 1
            predict_good += (predict_class_index == class_index)

    # Расчет и вывод итогов классификации
    good_ratio = round(predict_good / predict_all * 100., 2)
    print(f'=== Обработано образцов: {predict_all}, из них распознано верно: {predict_good}, доля верных: {good_ratio}% ===')

    # Построение матрицы ошибок без нормализации, покажет попадания в штуках
    cm = confusion_matrix(y_true, y_pred)

    # Отрисовка матрицы ошибок
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.set_title('Матрица ошибок по файлам аудио (не нормализованная)')
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
    disp.plot(ax=ax)
    plt.show()

```

Рисунок 1.7 – Функция `classify_test_files`

1.1.8. Постановка задания и форма данных.

Проверили форму данных обучающей и проверочной выборок.

```

# Ваше решение
print('Форма обучающей выборки:')
print(' x_train:', x_train.shape)
print(' y_train:', y_train.shape)

print('Форма проверочной выборки:')
print(' x_val: ', x_val.shape)
print(' y_val: ', y_val.shape)

Форма обучающей выборки:
x_train: (810, 37)
y_train: (810, 10)
Форма проверочной выборки:
x_val: (90, 37)
y_val: (90, 10)

```

Рисунок 1.8 – Постановка задания и форма данных

1.1.9. Архитектура полносвязной модели (Keras).

Реализована полносвязная сеть: `Dense(64, relu, input_shape) → Dropout(0.3) → Dense(32, relu) → Dropout(0.3) → Dense(16, relu) → Dropout(0.2) → BatchNormalization → Dense(10, softmax)`. Всего параметров: 5 274 (обучаемых: 5 242, необучаемых: 32).

```

# Ваше решение
model = Sequential()

# Полносвязный слой на 64 нейрона + Dropout 38%
model.add(Dense(64, activation='relu', input_shape=(x_train.shape[1],)))
model.add(Dropout(0.3))

# Полносвязный слой на 32 нейрона + Dropout 38%
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

# Полносвязный слой на 16 нейронов + Dropout 28%
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.2))

# Слой пакетной нормализации
model.add(BatchNormalization())

# Финальный слой классификатора с softmax
model.add(Dense(CLASS_COUNT, activation='softmax'))

```

Рисунок 1.9 – Код архитектуры полносвязной модели Keras

1.1.10. Компиляция и сводка архитектуры.

```

# Ваше решение
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,432
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dropout_2 (Dropout)	(None, 16)	0
batch_normalization (BatchNormalization)	(None, 16)	64
dense_3 (Dense)	(None, 10)	170

Total params: 5,274 (20.60 KB)
 Trainable params: 5,242 (20.48 KB)
 Non-trainable params: 32 (128.00 B)

Рисунок 1.10 – Компиляция и summary модели Keras

1.1.11. Обучение модели.

Обучение на `x_train/y_train`, `batch_size=32`, `epochs=1000`, `validation_data=(x_val, y_val)`. К последним эпохам (987–1000) точность на обучающей выборке ~69–71%, на валидационной ~62–63%. Функция потерь продолжает убывать.

```
## Ваше решение
history = model.fit(x_train, y_train,
                    batch_size=32,
                    epochs=1000,
                    validation_data=(x_val, y_val),
                    verbose=1)

show_history(history)
```

```
Epoch 987/1000      26/26      0s 6ms/step - accuracy: 0.7148 - loss: 0.8482 - val_accuracy: 0.6222 - val_loss: 1.0589
Epoch 988/1000      26/26      0s 6ms/step - accuracy: 0.7160 - loss: 0.8839 - val_accuracy: 0.6222 - val_loss: 1.0544
Epoch 989/1000      26/26      0s 6ms/step - accuracy: 0.6988 - loss: 0.8856 - val_accuracy: 0.6333 - val_loss: 1.0515
Epoch 990/1000      26/26      0s 6ms/step - accuracy: 0.6938 - loss: 0.8866 - val_accuracy: 0.6222 - val_loss: 1.0569
Epoch 991/1000      26/26      0s 6ms/step - accuracy: 0.6914 - loss: 0.9018 - val_accuracy: 0.6222 - val_loss: 1.0576
Epoch 992/1000      26/26      0s 6ms/step - accuracy: 0.6864 - loss: 0.9676 - val_accuracy: 0.6333 - val_loss: 1.0577
Epoch 993/1000      26/26      0s 6ms/step - accuracy: 0.7173 - loss: 0.8708 - val_accuracy: 0.6333 - val_loss: 1.0580
Epoch 994/1000      26/26      0s 6ms/step - accuracy: 0.6852 - loss: 0.9031 - val_accuracy: 0.6333 - val_loss: 1.0588
Epoch 995/1000      26/26      0s 6ms/step - accuracy: 0.6877 - loss: 0.9100 - val_accuracy: 0.6333 - val_loss: 1.0579
Epoch 996/1000      26/26      0s 6ms/step - accuracy: 0.6827 - loss: 0.9350 - val_accuracy: 0.6333 - val_loss: 1.0572
Epoch 997/1000      26/26      0s 7ms/step - accuracy: 0.6741 - loss: 0.9188 - val_accuracy: 0.6222 - val_loss: 1.0563
Epoch 998/1000      26/26      0s 6ms/step - accuracy: 0.6852 - loss: 0.9074 - val_accuracy: 0.6222 - val_loss: 1.0545
Epoch 999/1000      26/26      0s 6ms/step - accuracy: 0.6901 - loss: 0.9081 - val_accuracy: 0.6222 - val_loss: 1.0570
Epoch 1000/1000     26/26      0s 6ms/step - accuracy: 0.6864 - loss: 0.8633 - val_accuracy: 0.6222 - val_loss: 1.0575
```

Рисунок 1.11 – Код обучения и лог последних эпох (987–1000)

На графике точности обе кривые монотонно растут: обучающая достигает ~70%, валидационная ~62%. На графике потерь валидационная ошибка стабилизируется выше обучающей – умеренное переобучение. Кривые не сошлись, что указывает на потенциал дальнейшего улучшения.

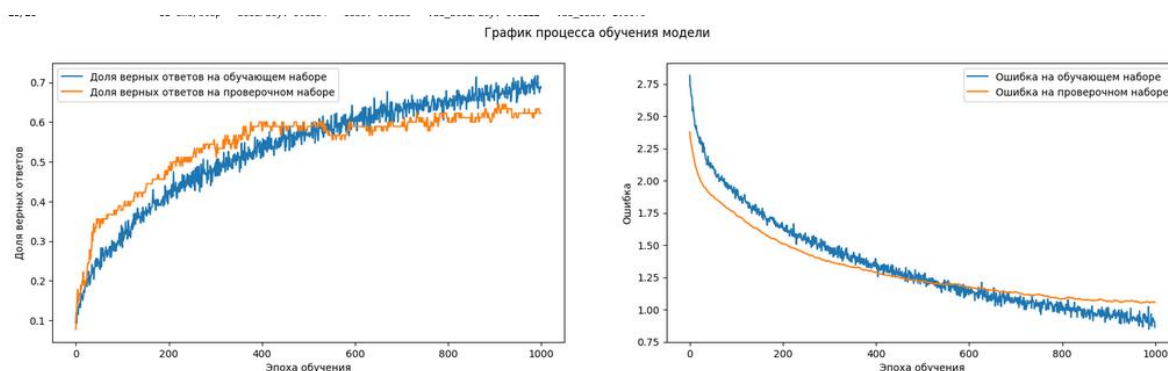


Рисунок 1.12 – Графики точности и потерь за 1000 эпох (Keras)

1.1.13. Проверка на тестовых файлах.

`classify_test_files(from_index=90, n_files=10)`: обработано 100 образцов, верно распознано 56, доля верных 56.0%. Лучшие результаты: jazz (10/10) и рор (10/10). Наибольшие трудности: country и rock вследствие акустической близости к другим жанрам.

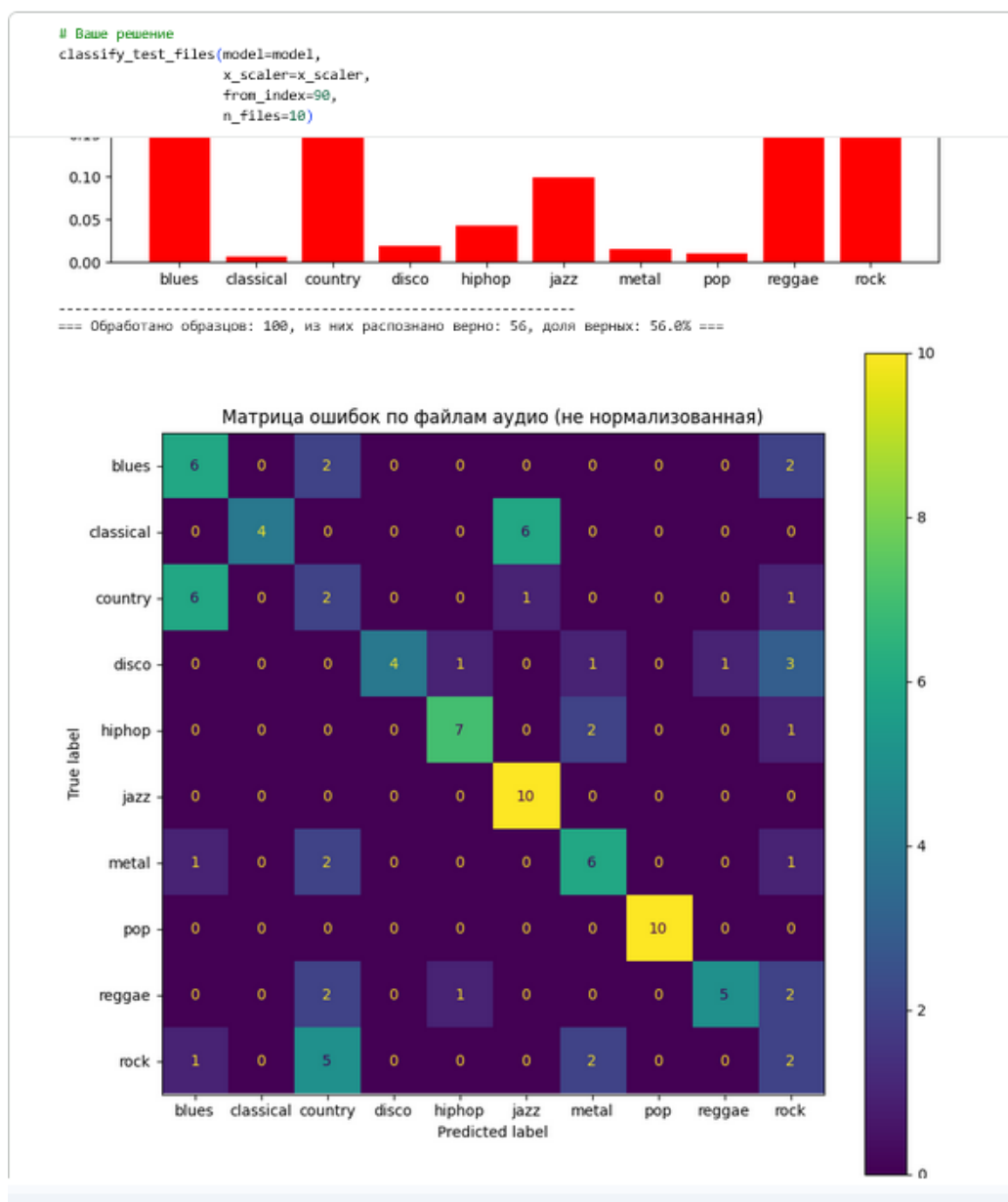


Рисунок 1.13 – Результаты и матрица ошибок (Keras)

1.2. Реализация на PyTorch.

1.2.1. Импорт библиотек PyTorch и подготовка данных.

Дополнительно к ранее подготовленным данным импортируются: torch, torch.nn, torch.optim, TensorDataset и DataLoader из torch.utils.data. Определяется устройство: cuda (если доступен GPU) или cpu. Фиксируется seed для воспроизводимости: torch.manual_seed(42), np.random.seed(42). Признаки извлекаются тем же способом, что в задании 1.1 (extract_data, файлы 0–89). Форма x_train_data: (900, 37). Нормировка: StandardScaler. Разбиение train_test_split со stratify: x_train (810, 37), x_val (90, 37).

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import numpy as np # Import numpy

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Устройство:', device)

# Фиксируем seed для воспроизводимости
torch.manual_seed(42)
np.random.seed(42)

... Устройство: cuda
```

Подготовка данных

Извлекаем признаки из аудиофайлов тем же способом, что использует `classify_test_files` — через функцию `extract_data`.

```
# Извлекаем признаки из обучающих файлов (0..89 каждого класса)
x_train_data, y_train_data = extract_data(file_index_start=0,
                                          file_index_end=FILE_INDEX_TRAIN_SPLIT,
                                          duration_sec=DURATION_SEC)

print('Форма x_train_data:', x_train_data.shape)
print('Форма y_train_data:', y_train_data.shape)

# Нормировка признаков
x_scaler = StandardScaler()
x_train_data_scaled = x_scaler.fit_transform(x_train_data)

# Разбиение на train/val
x_train, x_val, y_train, y_val = train_test_split(
    x_train_data_scaled,
    y_train_data,
    stratify=y_train_data,
    test_size=VALIDATION_SPLIT,
    random_state=42
)

Жанр blues готов -> 128 c
Жанр classical готов -> 111 c
Жанр country готов -> 112 c
Жанр disco готов -> 111 c
Жанр hip-hop готов -> 111 c
Жанр jazz готов -> 112 c
Жанр metal готов -> 111 c
Жанр pop готов -> 113 c
Жанр reggae готов -> 111 c
Жанр rock готов -> 111 c
Форма x_train_data: (900, 37)
Форма y_train_data: (900, 10)
```

Рисунок 1.14 – Импорт PyTorch, подготовка данных и извлечение признаков

1.2.2. Преобразование в тензоры и создание DataLoader.

Метки `y_train` переводятся из one-hot в индексы классов через `np.argmax` (если `ndim==2`). Данные конвертируются в тензоры PyTorch: `X_train_t` (float32), `y_train_t` (long), `X_val_t`, `y_val_t`. Формируются `DataLoader`: `train_loader` с `batch_size=32` и `shuffle=True`, `val_loader` с `shuffle=False`. Форма тензоров: `X_train_t torch.Size([810, 37])`, `X_val_t torch.Size([90, 37])`.

Задание 1. Проверка формы данных

```
print('Форма обучающей выборки:')
print(' x_train:', x_train.shape)
print(' y_train:', y_train.shape)

print('Форма проверочной выборки:')
print(' x_val: ', x_val.shape)
print(' y_val: ', y_val.shape)
```

```
--- Форма обучающей выборки:
    x_train: (810, 37)
    y_train: (810, 10)
Форма проверочной выборки:
    x_val:   (90, 37)
    y_val:   (90, 10)
```

Преобразование в тензоры PyTorch и создание DataLoader

```
# y_train приходит уже в one-hot формате из extract_data + переводим в индексы классов
if y_train.ndim == 2:
    y_train_idx = np.argmax(y_train, axis=1)
    y_val_idx   = np.argmax(y_val, axis=1)
else:
    y_train_idx = y_train.astype(np.int64)
    y_val_idx   = y_val.astype(np.int64)

# Тензоры
X_train_t = torch.tensor(x_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train_idx, dtype=torch.long)
X_val_t   = torch.tensor(x_val, dtype=torch.float32)
y_val_t   = torch.tensor(y_val_idx, dtype=torch.long)

print('X_train_t:', X_train_t.shape, ' y_train_t:', y_train_t.shape)
print('X_val_t: ', X_val_t.shape, ' y_val_t: ', y_val_t.shape)

# DataLoader
BATCH_SIZE = 32
train_loader = DataLoader(TensorDataset(X_train_t, y_train_t),
                          batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(TensorDataset(X_val_t, y_val_t),
                          batch_size=BATCH_SIZE, shuffle=False)
```

```
X_train_t: torch.Size([810, 37]) y_train_t: torch.Size([810])
X_val_t:   torch.Size([90, 37]) y_val_t:   torch.Size([90])
```

Рисунок 1.15 – Проверка форм, преобразование в тензоры и создание DataLoader

1.2.3. Архитектура модели (nn.Module).

Класс `AudioClassifier` наследует `nn.Module`. Архитектура аналогична заданию 1.1: `fc1 Linear(37→64) → drop1 Dropout(0.3) → fc2 Linear(64→32) → drop2 Dropout(0.3) → fc3 Linear(32→16) → drop3 Dropout(0.2) → bn BatchNorm1d(16) → fc_out Linear(16→10)`. В PyTorch softmax встроен в `CrossEntropyLoss`, поэтому в `forward` выходной слой возвращает логиты. Модель переносится на выбранное устройство: `.to(device)`. Всего параметров: 5 242.

```

class AudioClassifier(nn.Module):
    def __init__(self, input_dim, n_classes):
        super().__init__()
        # Полносвязный слой на 64 нейрона + Dropout 30%
        self.fc1 = nn.Linear(input_dim, 64)
        self.drop1 = nn.Dropout(0.3)

        # Полносвязный слой на 32 нейрона + Dropout 30%
        self.fc2 = nn.Linear(64, 32)
        self.drop2 = nn.Dropout(0.3)

        # Полносвязный слой на 16 нейронов + Dropout 20%
        self.fc3 = nn.Linear(32, 16)
        self.drop3 = nn.Dropout(0.2)

        # Слой пакетной нормализации
        self.bn = nn.BatchNorm1d(16)

        # Финальный полносвязный слой (логиты, softmax внутри CrossEntropyLoss)
        self.fc_out = nn.Linear(16, n_classes)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.drop1(x)
        x = torch.relu(self.fc2(x))
        x = self.drop2(x)
        x = torch.relu(self.fc3(x))
        x = self.drop3(x)
        x = self.bn(x)
        x = self.fc_out(x)
        return x

model = AudioClassifier(input_dim=x_train.shape[1], n_classes=CLASS_COUNT).to(device)
print(model)

AudioClassifier(
  (fc1): Linear(in_features=37, out_features=64, bias=True)
  (drop1): Dropout(p=0.3, inplace=False)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (drop2): Dropout(p=0.3, inplace=False)
  (fc3): Linear(in_features=32, out_features=16, bias=True)
  (drop3): Dropout(p=0.2, inplace=False)
  (bn): BatchNorm1d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc_out): Linear(in_features=16, out_features=10, bias=True)
)

```

Рисунок 1.16 – Класс AudioClassifier (nn.Module) и вывод архитектуры

1.2.4. Оптимизатор, функция потерь и сводка параметров.

Функция потерь: `nn.CrossEntropyLoss` (включает `LogSoftmax` + `NLLLoss`). Оптимизатор: `optim.Adam(lr=0.0001)`. Подсчёт параметров вручную (аналог `model.summary()`): всего параметров 5 242, обучаемых 5 242. Архитектура слоёв: fc1 Linear (2432), fc2 Linear (2080), fc3 Linear (528), bn BatchNorm1d (32), fc_out Linear (170).

Задание 3. Оптимизатор, функция потерь и сводка модели

```
1  # CrossEntropyLoss = LogSoftmax + NLLLoss (включает softmax внутри)
2  criterion = nn.CrossEntropyLoss()
3  optimizer = optim.Adam(model.parameters(), lr=0.0001)
4
5  # Аналог model.summary() из Keras
6  total_params = sum(p.numel() for p in model.parameters())
7  trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
8  print(f'Всего параметров: {total_params}')
9  print(f'Обучаемых параметров: {trainable_params}')
10 print()
11 print('Архитектура слоёв:')
12 for name, layer in model.named_modules():
13     if list(layer.children()) == [] and name != '':
14         n = sum(p.numel() for p in layer.parameters())
15         print(f' {name:10s} {layer.__class__.__name__:18s} params={n}')
```

Всего параметров: 5242
Обучаемых параметров: 5242

Архитектура слоёв:

fc1	Linear	params=2432
drop1	Dropout	params=0
fc2	Linear	params=2080
drop2	Dropout	params=0
fc3	Linear	params=528
drop3	Dropout	params=0
bn	BatchNorm1d	params=32
fc_out	Linear	params=170

Рисунок 1.17 – Оптимизатор, функция потерь и сводка параметров

1.2.5. Цикл обучения.

В PyTorch нет встроенного `.fit()` – обучение реализовано вручную. Цикл на 1000 эпох: на каждой эпохе выполняется проход по `train_loader` с обновлением весов (`model.train()`, `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`) и проход по `val_loader` без обновления (`model.eval()`, `torch.no_grad()`). Метрики accuracy и loss сохраняются в словарь `history`. Вывод каждые 50 эпох и на последней.

```

EPOCHS = 1000

history = {'accuracy': [], 'val_accuracy': [], 'loss': [], 'val_loss': []}

for epoch in range(EPOCHS):
    # === Обучение ===
    model.train()
    train_loss, train_correct, train_total = 0.0, 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

        train_loss += loss.item() * xb.size(0)
        train_correct += (logits.argmax(dim=1) == yb).sum().item()
        train_total += xb.size(0)

    train_loss /= train_total
    train_acc = train_correct / train_total

    # === Валидация ===
    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)
            val_loss += loss.item() * xb.size(0)
            val_correct += (logits.argmax(dim=1) == yb).sum().item()
            val_total += xb.size(0)

    val_loss /= val_total
    val_acc = val_correct / val_total

    history['accuracy'].append(train_acc)
    history['val_accuracy'].append(val_acc)
    history['loss'].append(train_loss)
    history['val_loss'].append(val_loss)

    if (epoch + 1) % 50 == 0 or epoch == 0:
        print(f'Эпоха {epoch+1:4d}/{EPOCHS} | '
              f'train_loss={train_loss:.4f} train_acc={train_acc:.4f} | '
              f'val_loss={val_loss:.4f} val_acc={val_acc:.4f}')

print('\nОбучение завершено!')

```

Рисунок 1.18 – Код цикла обучения PyTorch

1.2.6. Лог обучения.

Прогресс обучения выводится каждые 50 эпох. К эпохе 1000: $\text{train_loss} \approx 0.72$, $\text{train_acc} \approx 0.76$, $\text{val_loss} \approx 1.14$, $\text{val_acc} \approx 0.64$. Динамика соответствует результатам Keras: валидационная точность достигает ~64%, тренировочная ~76% – умеренное переобучение.

Эпоха	1/1000	train_loss=2.4538	train_acc=0.0975	val_loss=2.3187	val_acc=0.0889
Эпоха	50/1000	train_loss=1.9007	train_acc=0.2963	val_loss=1.7861	val_acc=0.3667
Эпоха	100/1000	train_loss=1.6730	train_acc=0.4123	val_loss=1.5706	val_acc=0.4778
Эпоха	150/1000	train_loss=1.5188	train_acc=0.4642	val_loss=1.4284	val_acc=0.5333
Эпоха	200/1000	train_loss=1.3884	train_acc=0.5198	val_loss=1.3402	val_acc=0.5556
Эпоха	250/1000	train_loss=1.3170	train_acc=0.5432	val_loss=1.2259	val_acc=0.6000
Эпоха	300/1000	train_loss=1.1826	train_acc=0.5617	val_loss=1.1982	val_acc=0.6222
Эпоха	350/1000	train_loss=1.1311	train_acc=0.6123	val_loss=1.1232	val_acc=0.6333
Эпоха	400/1000	train_loss=1.0652	train_acc=0.6370	val_loss=1.0964	val_acc=0.6667
Эпоха	450/1000	train_loss=1.0036	train_acc=0.6593	val_loss=1.0801	val_acc=0.6556
Эпоха	500/1000	train_loss=0.9849	train_acc=0.6741	val_loss=1.0728	val_acc=0.6222
Эпоха	550/1000	train_loss=0.9352	train_acc=0.6679	val_loss=1.0429	val_acc=0.6556
Эпоха	600/1000	train_loss=0.9227	train_acc=0.6790	val_loss=1.0357	val_acc=0.6444
Эпоха	650/1000	train_loss=0.8548	train_acc=0.6951	val_loss=1.0430	val_acc=0.6556
Эпоха	700/1000	train_loss=0.8527	train_acc=0.7123	val_loss=1.0496	val_acc=0.6556
Эпоха	750/1000	train_loss=0.8371	train_acc=0.7185	val_loss=1.0602	val_acc=0.6778
Эпоха	800/1000	train_loss=0.7689	train_acc=0.7457	val_loss=1.0951	val_acc=0.6556
Эпоха	850/1000	train_loss=0.8189	train_acc=0.7272	val_loss=1.0896	val_acc=0.6556
Эпоха	900/1000	train_loss=0.7183	train_acc=0.7753	val_loss=1.1080	val_acc=0.6556
Эпоха	950/1000	train_loss=0.6899	train_acc=0.7778	val_loss=1.1470	val_acc=0.6556
Эпоха	1000/1000	train_loss=0.7226	train_acc=0.7605	val_loss=1.1354	val_acc=0.6444

Обучение завершено!

Рисунок 1.19 – Лог обучения (эпохи 1–1000, шаг 50)

1.2.7. Графики процесса обучения.

Графики строятся по сохранённому словарию history. На кривой точности обучающая выборка достигает ~76–80%, валидационная – ~65–68%. На кривой потерь характерный всплеск в начале, затем монотонное убывание. Поведение кривых аналогично Keras: модели сходятся к схожим показателям качества.

Графики точности и ошибки

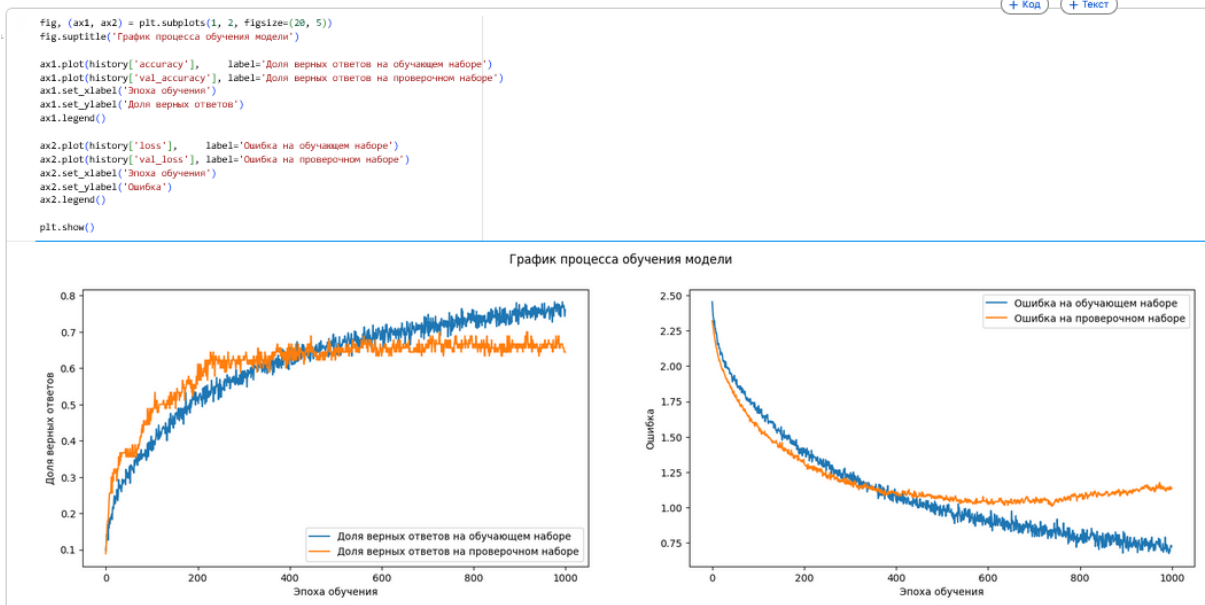


Рисунок 1.20 – Графики точности и потерь за 1000 эпох (PyTorch)

1.2.8. Обёртка KerasLikeWrapper и тест на тестовых файлах.

Поскольку функция classify_test_files написана под Keras API (ожидает .predict()), создаётся класс-обёртка KerasLikeWrapper с методом predict,

который переводит данные в тензор, пропускает через модель, применяет softmax и возвращает numpy-массив вероятностей. Затем вызывается `classify_test_files` с `model_wrapped`, `x_scaler`, `from_index=90`, `n_files=10`.

```
class KerasLikeWrapper:
    """Обёртка над PyTorch-моделью, имитирующая model.predict() из Keras."""
    def __init__(self, torch_model, device):
        self.model = torch_model
        self.device = device

    def predict(self, x, verbose=0):
        self.model.eval()
        with torch.no_grad():
            x_t = torch.tensor(x, dtype=torch.float32).to(self.device)
            logits = self.model(x_t)
            probs = torch.softmax(logits, dim=1).cpu().numpy()
        return probs

model_wrapped = KerasLikeWrapper(model, device)

# Запуск встроенной функции тестирования из исходного ноутбука
classify_test_files(model=model_wrapped,
                    x_scaler=x_scaler,
                    from_index=90,
                    n_files=10)
```

Рисунок 1.21 – Класс KerasLikeWrapper и вызов `classify_test_files`

1.2.9. Результаты и матрица ошибок.

Обработано 100 образцов, верно распознано 61, доля верных: 61.0%. Результат превышает показатель Keras (56%). Матрица ошибок показывает: поп (10/10), hip-hop (8/10), jazz (9/10) распознаются хорошо. Наибольшие трудности: blues (4/10), country (4/10) и rock (3/10), которые путаются с соседними жанрами.

=== Обработано образцов: 100, из них распознано верно: 61, доля верных: 61.0% ===

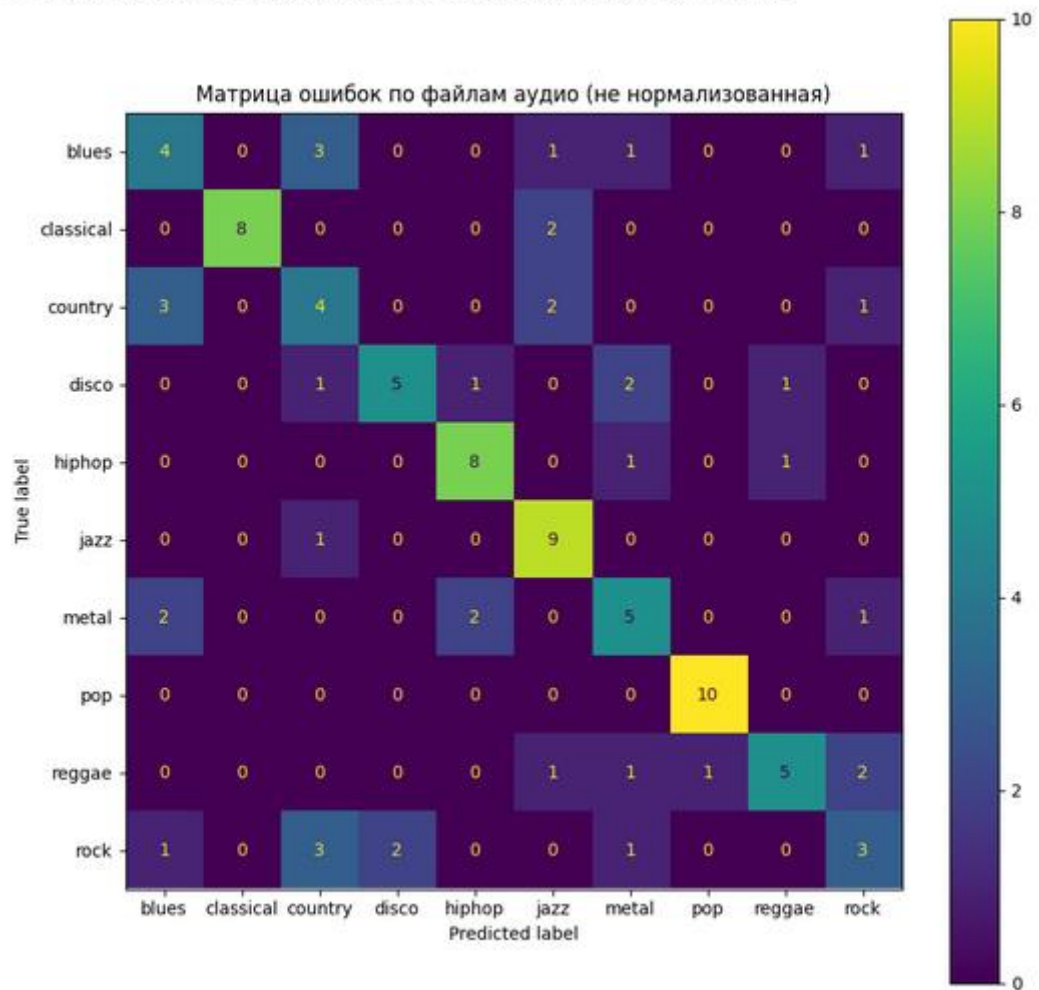


Рисунок 1.22 – Результаты классификации и матрица ошибок (PyTorch)

2. Решение задачи классификации жанров с помощью Conv1D.

2.1. Импорт библиотек и настройка окружения.

На первом шаге выполняется импорт всех необходимых библиотек: NumPy для работы с массивами, Matplotlib для визуализации, Librosa для параметризации аудиосигналов, а также соответствующих модулей TensorFlow/Keras для построения и обучения нейронных сетей. Дополнительно подключаются утилиты sklearn для разбиения данных, кодирования меток и оценки качества модели.

```

1: # Массивы
import numpy as np

# Отрисовка графиков
import matplotlib.pyplot as plt

# Загрузка из google облака
import gdown

# Преобразование категориальных данных в one hot encoding
from tensorflow.keras.utils import to_categorical

# Работа с папками и файлами
import os

# Утилиты работы со временем
import time

# Работа со случайными числами
import random

# Математические функции
import math

# Сохранение и загрузка структур данных Python
import pickle

# Параметризация аудио
import librosa

# Оптимизаторы для обучения моделей
from tensorflow.keras.optimizers import Adam, RMSprop

# Конструирование и загрузка моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model, load_model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, Conv2D, LSTM
from tensorflow.keras.layers import MaxPooling1D, AveragePooling1D, SpatialDropout1D

# Разбиение на обучающую и проверочную выборку
from sklearn.model_selection import train_test_split

# Кодирование категориальных меток, нормирование числовых данных
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline

```

Рисунок 2.1 – Импорт библиотек

2.2. Загрузка и распаковка датасета.

Датасет жанров музыки (GTZAN) загружается из облачного хранилища с помощью библиотеки gdown. После загрузки архив распаковывается, и проверяется структура папок: 10 жанровых классов (blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, rock), каждый содержит 100 аудиофайлов формата .au.

Загрузим датасет в Colab:

```
In [ ]: # Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l12/genres.zip', None, quiet=True)

Out[ ]: 'genres.zip'

In [ ]: # Распаковка архива на локальный диск colab
!unzip -qo genres.zip

# Проверка выгруженных папок
!ls genres

# Проверка содержимого одной папки
!ls genres/blues

blues classical country disco hiphop jazz metal pop reggae rock
blues.00000.au blues.00020.au blues.00040.au blues.00060.au blues.00080.au
blues.00001.au blues.00021.au blues.00041.au blues.00061.au blues.00081.au
blues.00002.au blues.00022.au blues.00042.au blues.00062.au blues.00082.au
blues.00003.au blues.00023.au blues.00043.au blues.00063.au blues.00083.au
blues.00004.au blues.00024.au blues.00044.au blues.00064.au blues.00084.au
blues.00005.au blues.00025.au blues.00045.au blues.00065.au blues.00085.au
blues.00006.au blues.00026.au blues.00046.au blues.00066.au blues.00086.au
blues.00007.au blues.00027.au blues.00047.au blues.00067.au blues.00087.au
blues.00008.au blues.00028.au blues.00048.au blues.00068.au blues.00088.au
blues.00009.au blues.00029.au blues.00049.au blues.00069.au blues.00089.au
blues.00010.au blues.00030.au blues.00050.au blues.00070.au blues.00090.au
blues.00011.au blues.00031.au blues.00051.au blues.00071.au blues.00091.au
blues.00012.au blues.00032.au blues.00052.au blues.00072.au blues.00092.au
blues.00013.au blues.00033.au blues.00053.au blues.00073.au blues.00093.au
blues.00014.au blues.00034.au blues.00054.au blues.00074.au blues.00094.au
blues.00015.au blues.00035.au blues.00055.au blues.00075.au blues.00095.au
blues.00016.au blues.00036.au blues.00056.au blues.00076.au blues.00096.au
blues.00017.au blues.00037.au blues.00057.au blues.00077.au blues.00097.au
blues.00018.au blues.00038.au blues.00058.au blues.00078.au blues.00098.au
blues.00019.au blues.00039.au blues.00059.au blues.00079.au blues.00099.au
```

Рисунок 2.2 – Загрузка и просмотр содержимого датасета

2.3. Параметры датасета и аудио.

Задаются основные параметры обработки аудиофайлов: общая длительность анализируемого фрагмента (30 сек), длина окна скользящего анализа (5 сек), шаг скользящего окна (1 сек). Также задаются параметры извлечения признаков Librosa: размер окна БПФ ($N_{\text{FFT}}=2048$) и шаг хоппинга ($\text{HOP_LENGTH}=512$). Файлы с 0 по 89 используются для обучения/валидации, файлы 90–99 – для тестирования.

```

In [ ]: # Ваше решение

# Папка с файлами датасета
FILE_DIR = './genres'
# Список классов
CLASS_LIST = os.listdir(FILE_DIR)
CLASS_LIST.sort()
CLASS_COUNT = len(CLASS_LIST)

# Файлы и их разбиение
CLASS_FILES = 100 # всего файлов в каждом классе
FILE_INDEX_TRAIN_SPLIT = 90 # первые 90 → обучение/валидация, последние 10 → тест
VALIDATION_SPLIT = 0.1

# Параметры аудио и окна
DURATION_SEC = 30 # анализируемая длительность файла
WINDOW_SEC = 5 # длина одного отрезка (окна)
STEP_SEC = 1 # шаг скользящего окна

# Параметры извлечения признаков Librosa
N_FFT = 2048
HOP_LENGTH = 512

print('Классы:', CLASS_LIST)
print('Количество классов:', CLASS_COUNT)

Классы: ['blues', 'classical', 'country', 'disco', 'hiphop', 'jazz', 'metal', 'pop', 'reggae', 'rock']
Количество классов: 10

```

Рисунок 2.3 – Параметры датасета и аудио

2.4. Функции извлечения признаков.

Для каждого аудиоотрезка вычисляется набор акустических признаков без усреднения по времени – в виде временных последовательностей векторов:

- Хромограмма (12 компонент)
- MFCC – мел-кепстральные коэффициенты (20 компонент)
- RMSE – среднеквадратичная энергия (1 компонент)
- Спектральный центрoид (1 компонент)
- Ширина полосы частот (1 компонент)
- Спектральный rolloff (1 компонент)
- Пересечения нуля (1 компонент)

Все признаки подрезаются до одинаковой длины по оси времени и объединяются в матрицу формы (T, 37).


```

# Извлечение признаков для одного аудио-отрезка
def extract_features(y, sr, n_fft=N_FFT, hop_length=HOP_LENGTH):
    # Хромограмма (12, T)
    chroma = librosa.feature.chroma_stft(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # MFCC (20, T)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # RMSE (1, T)
    rmse = librosa.feature.rms(y=y, hop_length=hop_length)
    # Спектральный центроид (1, T)
    cent = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Ширина полосы частот (1, T)
    bw = librosa.feature.spectral_bandwidth(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Спектральный rolloff (1, T)
    roll = librosa.feature.spectral_rolloff(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Пересечения нуля (1, T)
    zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length)

    # Подрезаем все признаки до одинаковой длины по оси времени
    min_t = min(chroma.shape[1], mfcc.shape[1], rmse.shape[1],
                cent.shape[1], bw.shape[1], roll.shape[1], zcr.shape[1])

    feats = np.vstack([
        chroma[:, :min_t],
        mfcc[:, :min_t],
        rmse[:, :min_t],
        cent[:, :min_t],
        bw[:, :min_t],
        roll[:, :min_t],
        zcr[:, :min_t]
    ])
    # Возвращаем форму (время, признаки) – как «последовательность векторов»
    return feats.T.astype('float32')

```

Рисунок 2.4 – Функция извлечения признаков (без усреднения по времени)

2.5. Формирование последовательностей из аудиофайла.

Функция `file_to_sequences` применяет скользящее окно к каждому аудиофайлу: для каждого окна длиной 5 сек с шагом 1 сек извлекается матрица признаков. В результате из одного файла получается список последовательностей формы (T, F), где T – количество временных шагов, F=37 – количество признаков.

```

# Из одного файла получаем список последовательностей (по скользящему окну)
def file_to_sequences(file_path, window_sec=WINDOW_SEC, step_sec=STEP_SEC,
                     duration_sec=DURATION_SEC):
    y, sr = librosa.load(file_path, mono=True, duration=duration_sec)

    window_len = int(window_sec * sr)
    step_len = int(step_sec * sr)

    sequences = []
    start = 0
    while start + window_len <= len(y):
        segment = y[start:start + window_len]
        seq = extract_features(segment, sr)
        sequences.append(seq)
        start += step_len

    return sequences # список массивов (T, F)

```

Рисунок 2.5 – Функция скользящего окна для формирования последовательностей

2.6. Сборка датасета по всем классам.

Функция `build_dataset` обходит все классы и файлы в заданном диапазоне индексов, загружает аудио, извлекает последовательности признаков и

формирует итоговые массивы данных. Обучающая выборка (файлы 0–89) содержит 23 391 последовательность, тестовая (файлы 90–99) – 2 600 последовательностей. Каждая последовательность имеет форму (216, 37).

```

: # Сбор данных в массивы
def build_dataset(file_index_start, file_index_end):
    x_data, y_data = [], []
    curr_time = time.time()

    for class_index in range(CLASS_COUNT):
        class_name = CLASS_LIST[class_index]
        for file_index in range(file_index_start, file_index_end):
            file_name = f'{FILE_DIR}/{class_name}/{class_name}.{str(file_index).zfill(5)}.au'
            seqs = file_to_sequences(file_name)
            for s in seqs:
                x_data.append(s)
                y_data.append(class_index)
            print(f'Жанр {class_name} готов -> {round(time.time() - curr_time)} c')
            curr_time = time.time()

    return np.array(x_data, dtype='float32'), np.array(y_data, dtype='int32')

# Обучающе-валидационный набор (файлы 0..89)
x_trainval, y_trainval = build_dataset(0, FILE_INDEX_TRAIN_SPLIT)
# Тестовый набор (файлы 90..99)
x_test, y_test = build_dataset(FILE_INDEX_TRAIN_SPLIT, CLASS_FILES)

print('x_trainval:', x_trainval.shape) # (N, T, F)
print('x_test: ', x_test.shape)

Жанр blues готов -> 199 c
Жанр classical готов -> 182 c
Жанр country готов -> 180 c
Жанр disco готов -> 181 c
Жанр hiphop готов -> 180 c
Жанр jazz готов -> 181 c
Жанр metal готов -> 183 c
Жанр pop готов -> 182 c
Жанр reggae готов -> 190 c
Жанр rock готов -> 180 c
Жанр blues готов -> 17 c
Жанр classical готов -> 19 c
Жанр country готов -> 17 c
Жанр disco готов -> 17 c
Жанр hiphop готов -> 17 c
Жанр jazz готов -> 19 c
Жанр metal готов -> 17 c
Жанр pop готов -> 17 c
Жанр reggae готов -> 18 c
Жанр rock готов -> 17 c
x_trainval: (23391, 216, 37)
x_test: (2600, 216, 37)

```

Рисунок 2.6 – Сборка датасета и размерности массивов

2.7. Нормировка признаков и разбиение train/val.

StandardScaler применяется к двумерному представлению данных ($N \times T$, F). После нормировки данные разворачиваются обратно в трёхмерный тензор. One-hot кодирование применяется для меток классов. Затем с помощью train_test_split выборка разделяется на обучающую (21 051

последовательность) и валидационную (2 340 последовательности) с сохранением пропорций классов.

```
# Размеры
n_train, T, F = x_trainval.shape
n_test = x_test.shape[0]

# Нормировка (fit только на train, чтобы избежать утечки)
x_scaler = StandardScaler()
x_trainval_2d = x_trainval.reshape(-1, F)
x_scaler.fit(x_trainval_2d)

x_trainval_scaled = x_scaler.transform(x_trainval_2d).reshape(n_train, T, F)
x_test_scaled = x_scaler.transform(x_test.reshape(-1, F)).reshape(n_test, T, F)

# One-hot для меток
y_trainval_oh = to_categorical(y_trainval, CLASS_COUNT)
y_test_oh = to_categorical(y_test, CLASS_COUNT)

# Разбиение train/val
x_train, x_val, y_train, y_val = train_test_split(
    x_trainval_scaled, y_trainval_oh,
    stratify=y_trainval,
    test_size=VALIDATION_SPLIT,
    random_state=42
)

print('x_train:', x_train.shape, 'y_train:', y_train.shape)
print('x_val: ', x_val.shape, 'y_val: ', y_val.shape)
print('x_test: ', x_test_scaled.shape, 'y_test: ', y_test_oh.shape)

x_train: (21051, 216, 37) y_train: (21051, 10)
x_val: (2340, 216, 37) y_val: (2340, 10)
x_test: (2600, 216, 37) y_test: (2600, 10)
```

Рисунок 2.7 – Нормировка и разбиение train/val

2.8. Архитектура свёрточного классификатора Conv1D.

Модель построена по аналогии с текстовым CNN: три последовательных свёрточных блока с пулингом, затем полносвязная голова классификатора. Структура блоков:

- Блок 1: Conv1D(128, kernel=5) → BatchNorm → MaxPooling1D(2) → SpatialDropout1D(0.2)
- Блок 2: Conv1D(128, kernel=5) → BatchNorm → MaxPooling1D(2) → SpatialDropout1D(0.3)
- Блок 3: Conv1D(64, kernel=3) → BatchNorm → AveragePooling1D(2) → SpatialDropout1D(0.3)
- Голова: Flatten → Dense(64, relu) → Dropout(0.4) → Dense(10, softmax)

Компиляция: Adam (lr=1e-3), categorical_crossentropy, метрика accuracy.

Всего параметров: 243 082.

```

model = Sequential()

model.add(Input(shape=(x_train.shape[1], x_train.shape[2])))

# Блок 1
model.add(Conv1D(128, kernel_size=5, padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling1D(pool_size=2))
model.add(SpatialDropout1D(0.2))

# Блок 2
model.add(Conv1D(128, kernel_size=5, padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling1D(pool_size=2))
model.add(SpatialDropout1D(0.3))

# Блок 3
model.add(Conv1D(64, kernel_size=3, padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(AveragePooling1D(pool_size=2))
model.add(SpatialDropout1D(0.3))

# Голова классификатора
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(CLASS_COUNT, activation='softmax'))

model.compile(optimizer=Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

Рисунок 2.8 – Архитектура модели Conv1D

2.9. Сводная таблица архитектуры модели.

Ниже представлена сводка слоёв модели с указанием выходных форм и числа обучаемых параметров, полученная с помощью `model.summary()`.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 216, 128)	23,808
batch_normalization (BatchNormalization)	(None, 216, 128)	512
max_pooling1d (MaxPooling1D)	(None, 108, 128)	0
spatial_dropout1d (SpatialDropout1D)	(None, 108, 128)	0
conv1d_1 (Conv1D)	(None, 108, 128)	82,048
batch_normalization_1 (BatchNormalization)	(None, 108, 128)	512
max_pooling1d_1 (MaxPooling1D)	(None, 54, 128)	0
spatial_dropout1d_1 (SpatialDropout1D)	(None, 54, 128)	0
conv1d_2 (Conv1D)	(None, 54, 64)	24,640
batch_normalization_2 (BatchNormalization)	(None, 54, 64)	256
average_pooling1d (AveragePooling1D)	(None, 27, 64)	0
spatial_dropout1d_2 (SpatialDropout1D)	(None, 27, 64)	0
flatten (Flatten)	(None, 1728)	0
dense (Dense)	(None, 64)	110,656
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650
Total params: 243,882 (949.54 KB)		
Trainable params: 242,442 (947.04 KB)		
Non-trainable params: 640 (2.50 KB)		

Рисунок 2.9 – Summary модели: слои, формы, параметры

2.10. Обучение модели.

Модель обучается в течение 50 эпох с размером батча 64. Используются обучающая и валидационная выборки. Начиная с эпохи 22 точность на обучающей выборке превышает 92%, на валидационной – 97%. К концу обучения (эпоха 50) достигнуты следующие показатели: train_accurasy $\approx$ 96.2%, val_accurasy $\approx$ 98.9%, что свидетельствует об отличном обобщении модели.

```
history = model.fit(x_train, y_train,
                    batch_size=64,
                    epochs=50,
                    validation_data=(x_val, y_val),
                    verbose=1)
```

```
Epoch 22/50
329/329 — 2s 7ms/step - accuracy: 0.9255 - loss: 0.2329 - val_accuracy: 0.9761 - val_loss: 0.0732
Epoch 23/50
329/329 — 2s 7ms/step - accuracy: 0.9300 - loss: 0.2173 - val_accuracy: 0.9791 - val_loss: 0.0613
Epoch 24/50
329/329 — 2s 7ms/step - accuracy: 0.9331 - loss: 0.2097 - val_accuracy: 0.9774 - val_loss: 0.0732
Epoch 25/50
329/329 — 2s 7ms/step - accuracy: 0.9349 - loss: 0.2018 - val_accuracy: 0.9812 - val_loss: 0.0627
Epoch 26/50
329/329 — 3s 8ms/step - accuracy: 0.9378 - loss: 0.1946 - val_accuracy: 0.9846 - val_loss: 0.0459
Epoch 27/50
329/329 — 3s 8ms/step - accuracy: 0.9378 - loss: 0.1925 - val_accuracy: 0.9863 - val_loss: 0.0437
Epoch 28/50
329/329 — 2s 7ms/step - accuracy: 0.9430 - loss: 0.1783 - val_accuracy: 0.9752 - val_loss: 0.0716
Epoch 29/50
329/329 — 2s 7ms/step - accuracy: 0.9448 - loss: 0.1737 - val_accuracy: 0.9868 - val_loss: 0.0433
Epoch 30/50
329/329 — 2s 7ms/step - accuracy: 0.9428 - loss: 0.1767 - val_accuracy: 0.9791 - val_loss: 0.0589
Epoch 31/50
329/329 — 3s 8ms/step - accuracy: 0.9478 - loss: 0.1616 - val_accuracy: 0.9902 - val_loss: 0.0383
Epoch 32/50
329/329 — 3s 8ms/step - accuracy: 0.9474 - loss: 0.1664 - val_accuracy: 0.9927 - val_loss: 0.0272
Epoch 33/50
329/329 — 2s 7ms/step - accuracy: 0.9497 - loss: 0.1557 - val_accuracy: 0.9863 - val_loss: 0.0421
Epoch 34/50
329/329 — 2s 7ms/step - accuracy: 0.9495 - loss: 0.1611 - val_accuracy: 0.9885 - val_loss: 0.0456
Epoch 35/50
329/329 — 2s 7ms/step - accuracy: 0.9498 - loss: 0.1575 - val_accuracy: 0.9868 - val_loss: 0.0373
Epoch 36/50
329/329 — 3s 8ms/step - accuracy: 0.9547 - loss: 0.1432 - val_accuracy: 0.9868 - val_loss: 0.0322
Epoch 37/50
329/329 — 5s 7ms/step - accuracy: 0.9534 - loss: 0.1457 - val_accuracy: 0.9842 - val_loss: 0.0441
Epoch 38/50
329/329 — 3s 8ms/step - accuracy: 0.9520 - loss: 0.1496 - val_accuracy: 0.9876 - val_loss: 0.0334
Epoch 39/50
329/329 — 2s 7ms/step - accuracy: 0.9565 - loss: 0.1399 - val_accuracy: 0.9897 - val_loss: 0.0343
Epoch 40/50
329/329 — 3s 8ms/step - accuracy: 0.9608 - loss: 0.1278 - val_accuracy: 0.9893 - val_loss: 0.0328
Epoch 41/50
329/329 — 2s 7ms/step - accuracy: 0.9589 - loss: 0.1296 - val_accuracy: 0.9923 - val_loss: 0.0255
Epoch 42/50
329/329 — 2s 7ms/step - accuracy: 0.9552 - loss: 0.1407 - val_accuracy: 0.9902 - val_loss: 0.0293
Epoch 43/50
329/329 — 3s 8ms/step - accuracy: 0.9575 - loss: 0.1348 - val_accuracy: 0.9919 - val_loss: 0.0284
Epoch 44/50
329/329 — 2s 7ms/step - accuracy: 0.9602 - loss: 0.1270 - val_accuracy: 0.9893 - val_loss: 0.0311
Epoch 45/50
329/329 — 3s 8ms/step - accuracy: 0.9587 - loss: 0.1306 - val_accuracy: 0.9876 - val_loss: 0.0358
Epoch 46/50
329/329 — 3s 8ms/step - accuracy: 0.9613 - loss: 0.1219 - val_accuracy: 0.9910 - val_loss: 0.0253
Epoch 47/50
329/329 — 2s 7ms/step - accuracy: 0.9649 - loss: 0.1099 - val_accuracy: 0.9944 - val_loss: 0.0195
Epoch 48/50
329/329 — 2s 7ms/step - accuracy: 0.9641 - loss: 0.1166 - val_accuracy: 0.9915 - val_loss: 0.0296
Epoch 49/50
329/329 — 2s 7ms/step - accuracy: 0.9608 - loss: 0.1217 - val_accuracy: 0.9923 - val_loss: 0.0200
Epoch 50/50
329/329 — 3s 8ms/step - accuracy: 0.9620 - loss: 0.1202 - val_accuracy: 0.9889 - val_loss: 0.0360
```

Рисунок 2.10 – Лог обучения (эпохи 22–50)

2.11. Графики процесса обучения.

На графике точности видно, что валидационная кривая стабильно выше обучающей, что указывает на хорошую регуляризацию. График потерь демонстрирует монотонное убывание обеих кривых без признаков переобучения. Модель успешно сходится к минимуму функции потерь.

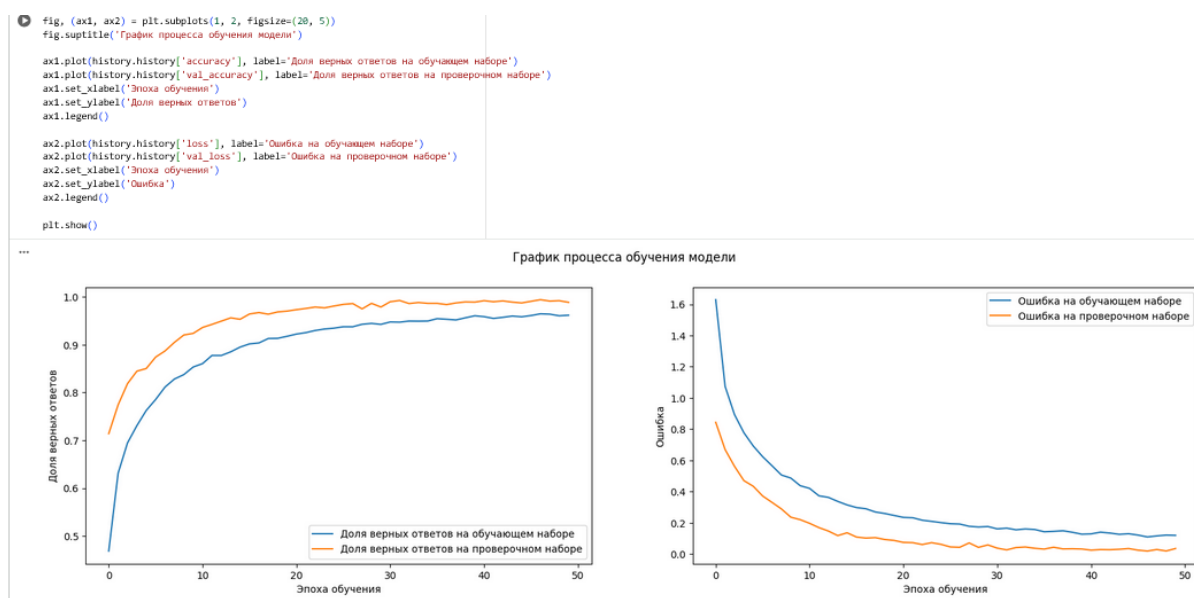


Рисунок 2.11 – Графики точности и потерь в процессе обучения

2.12. Оценка на тестовом наборе.

Тестирование выполняется двумя способами:

1. По отрезкам – `model.evaluate` на тестовых последовательностях.

Точность: 72.27%.

2. По файлам (голосование) – для каждого файла усредняются предсказания всех его 26 отрезков. Точность: 76.00%.

Точность на обучающей выборке составила 99.65%, что подтверждает высокую обучающую способность сети. Голосование по файлам улучшает результат по сравнению с оценкой по отдельным отрезкам.

```

# Оценка по отрезкам
test_loss, test_acc = model.evaluate(x_test_scaled, y_test_oh, verbose=0)
print(f'Точность по отрезкам: {test_acc * 100:.2f}%')

# Оценка по файлам (по 26 отрезков на файл при WINDOW_SEC=5, STEP_SEC=1, DURATION=30)
seqs_per_file = (DURATION_SEC - WINDOW_SEC) // STEP_SEC + 1
print(f'Отрезков на файл: {seqs_per_file}')

# Предсказание по всем тестовым отрезкам
y_pred_probs = model.predict(x_test_scaled, verbose=0)

# Группируем по файлам и усредняем
n_test_files = x_test_scaled.shape[0] // seqs_per_file
y_file_true, y_file_pred = [], []
for i in range(n_test_files):
    start = i * seqs_per_file
    end = start + seqs_per_file
    avg_probs = y_pred_probs[start:end].mean(axis=0)
    y_file_pred.append(np.argmax(avg_probs))
    y_file_true.append(y_test[start])

y_file_true = np.array(y_file_true)
y_file_pred = np.array(y_file_pred)

file_acc = (y_file_true == y_file_pred).mean()
print(f'Точность по файлам (голосование): {file_acc * 100:.2f}%')

# Точность на обучающем наборе
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
print(f'Точность на обучающей выборке: {train_acc * 100:.2f}%')

```

Точность по отрезкам: 72.27%
 Отрезков на файл: 26
 Точность по файлам (голосование): 76.00%
 Точность на обучающей выборке: 99.65%

Рисунок 2.12 – Оценка качества модели на тестовом наборе

2.13. Матрица ошибок по тестовым файлам.

Матрица ошибок показывает детализированные результаты классификации по 100 тестовым файлам (10 файлов $\times$ 10 классов). Наиболее уверенно классифицируются жанры *disco*, *jazz* и *pop* (10/10 верных предсказаний). Наибольшие трудности возникают с *blues* (4/10) и *rock* (2/10), что объясняется их схожестью с другими стилями.

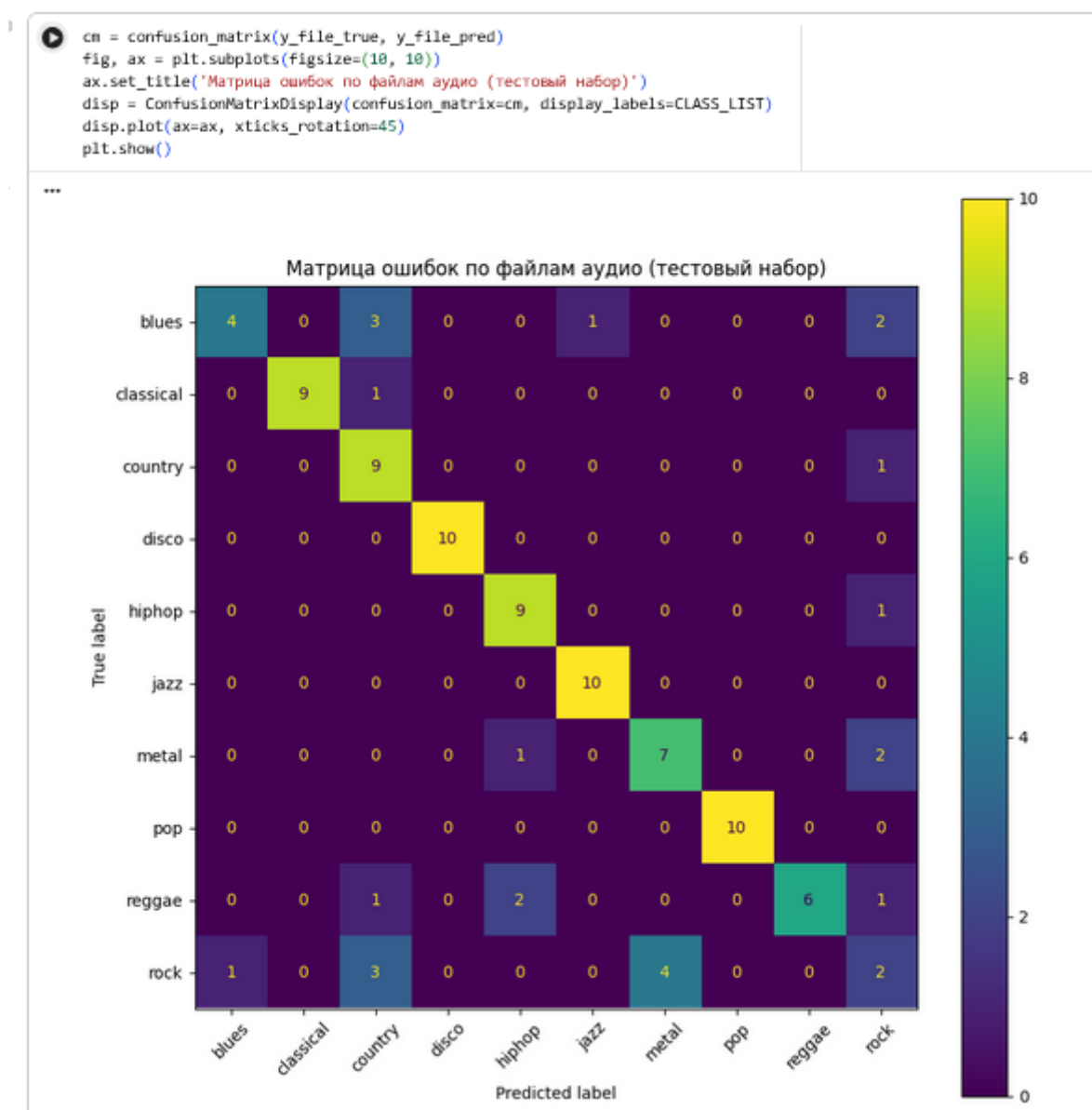


Рисунок 2.13 – Матрица ошибок классификации жанров по файлам (тестовый набор)

3. Решение задачи распознавания эмоций по голосу.

3.1. Импорт библиотек.

Для работы с задачей распознавания эмоций из речи выполняется импорт NumPy, Matplotlib, gdown, os, time, random, pickle, librosa и librosa.display. Из TensorFlow/Keras подключаются: to_categorical, оптимизатор Adam, Sequential и Model, слои Input, Dense, Dropout, BatchNormalization, Conv2D, MaxPooling2D, GlobalAveragePooling2D, а также колбэки EarlyStopping и ReduceLROnPlateau. Из sklearn – train_test_split, LabelEncoder, confusion_matrix, ConfusionMatrixDisplay и classification_report.

```

import numpy as np
import matplotlib.pyplot as plt
import gdown
import os
import time
import random
import pickle
import librosa
import librosa.display

from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (Input, Dense, Dropout, BatchNormalization,
                                     Flatten, Conv2D, MaxPooling2D, GlobalAveragePooling2D)
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

import warnings
warnings.filterwarnings('ignore')
%matplotlib inline

```

Рисунок 3.1 – Импорт библиотек для задачи распознавания эмоций

3.2. Загрузка датасета TESS.

Датасет Toronto Emotional Speech Set (TESS) загружается из облачного хранилища с помощью gdown. Архив распаковывается в директорию tess_data. TESS содержит записи двух актрис (OAF – Older Adult Female, YAF – Younger Adult Female), которые произносят слова с семью различными эмоциями: angry, disgust, fear, happy, neutral, pleasant_surprise, sad.

```

# Загрузка TESS
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/dataverse_files.zip',
               None, quiet=False)

# Распаковка
!unzip -qq dataverse_files.zip -d tess_data
!ls tess_data | head -20

```

Downloading...

From: https://storage.yandexcloud.net/aiueducation/Content/base/112/dataverse_files.zip

To: /content/dataverse_files.zip

100%|██████████| 224M/224M [00:16<00:00, 13.7MB/s]

MANIFEST.TXT

OAF_back_angry.wav

OAF_back_disgust.wav

OAF_back_fear.wav

OAF_back_happy.wav

OAF_back_neutral.wav

OAF_back_ps.wav

OAF_back_sad.wav

OAF_bar_angry.wav

OAF_bar_disgust.wav

OAF_bar_fear.wav

OAF_bar_happy.wav

OAF_bar_neutral.wav

OAF_bar_ps.wav

OAF_bar_sad.wav

OAF_base_angry.wav

OAF_base_disgust.wav

OAF_base_fear.wav

OAF_base_happy.wav

OAF_base_neutral.wav

Рисунок 3.2 – Загрузка и распаковка датасета TESS

3.3. Разбор структуры TESS.

Имя каждого wav-файла кодирует эмоцию: например, OAF_back_angry.wav или YAF_back_angry.wav. Последнее слово после символа «_» в имени файла соответствует эмоции. Реализована функция `parse_emotion_tess`, которая извлекает эмоцию и нормализует её к единому словарю (`ps` → `pleasant_surprise`). Всего найдено 2 800 wav-файлов: по 400 образцов на каждую из 7 эмоций.

```
# Поиск всех .wav файлов
import glob

tess_files = []
for root, dirs, files in os.walk('tess_data'):
    for f in files:
        if f.lower().endswith('.wav'):
            tess_files.append(os.path.join(root, f))

print(f'Найдено wav-файлов: {len(tess_files)}')
print('Примеры:')
for p in tess_files[:5]:
    print(' ', p)

# Извлечение метки из имени файла
def parse_emotion_tess(filepath):
    name = os.path.basename(filepath).lower().replace('.wav', '')
    # Эмоция – последнее слово после последнего '_'
    parts = name.split('_')
    emotion = parts[-1]
    # Нормализация
    mapping = {
        'ps': 'pleasant_surprise',
        'surprise': 'pleasant_surprise',
        'surprised': 'pleasant_surprise',
        'angry': 'angry',
        'disgust': 'disgust',
        'fear': 'fear',
        'happy': 'happy',
        'neutral': 'neutral',
        'sad': 'sad'
    }
    return mapping.get(emotion, emotion)

# Собираем метки
tess_labels = [parse_emotion_tess(p) for p in tess_files]

# Распределение по эмоциям
from collections import Counter
print('\nРаспределение эмоций в TESS:')
for emo, cnt in sorted(Counter(tess_labels).items()):
    print(f' {emo}: {cnt}')

Найдено wav-файлов: 2800
Примеры:
tess_data/OAF_laugh_angry.wav
tess_data/YAF_kite_fear.wav
tess_data/OAF_gaze_ps.wav
tess_data/YAF_fail_sad.wav
tess_data/OAF_yearn_happy.wav

Распределение эмоций в TESS:
angry: 400
disgust: 400
fear: 400
happy: 400
neutral: 400
pleasant_surprise: 400
sad: 400
```

Рисунок 3.3 – Разбор структуры файлов TESS и распределение по эмоциям

3.4. Извлечение мел-спектрограмм.

Для каждого аудиофайла TESS извлекается мел-спектрограмма с параметрами: частота дискретизации $SR=22050$ Гц, длительность 3 секунды, 128 мел-полос, $N_FFT=2048$, $HOP_LENGTH=512$. Файлы дополняются нулями до фиксированной длины $TARGET_LEN$ или обрезаются. Спектрограмма переводится в логарифмическую шкалу (дБ). Итоговый тензор X_tess имеет форму $(2800, 128, 130)$ – N образцов $\times N_mels \times T_frames$. Метки кодируются LabelEncoder в 7 классов.

```
SR = 22050
DURATION = 3.0
N_MELS = 128
N_FFT = 2048
HOP_LENGTH = 512
TARGET_LEN = int(SR * DURATION)

def extract_melspec(filepath, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH):
    y, _ = librosa.load(filepath, sr=sr, duration=DURATION)
    # Дополняем нулями до фиксированной длины
    if len(y) < TARGET_LEN:
        y = np.pad(y, (0, TARGET_LEN - len(y)))
    else:
        y = y[:TARGET_LEN]
    # Мел-спектрограмма в дБ
    mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=n_mels,
                                         n_fft=n_fft, hop_length=hop_length)
    mel_db = librosa.power_to_db(mel, ref=np.max)
    return mel_db.astype('float32')

# Извлекаем для всех файлов TESS
print('Извлечение признаков TESS...')
t0 = time.time()
X_tess = []
for i, fp in enumerate(tess_files):
    X_tess.append(extract_melspec(fp))
    if (i + 1) % 500 == 0:
        print(f' Обработано: {i+1}/{len(tess_files)}, время: {time.time()-t0:.1f} c')

X_tess = np.array(X_tess)
print(f'Форма X_tess: {X_tess.shape}') # (N, n_mels, T)
print(f'Время извлечения: {time.time()-t0:.1f} c')

# Кодирование меток
le = LabelEncoder()
y_tess_int = le.fit_transform(tess_labels)
CLASSES = list(le.classes_)
N_CLASSES = len(CLASSES)
print(f'\nКлассы ({N_CLASSES}): {CLASSES}')
```

Извлечение признаков TESS...

Обработано: 500/2800, время: 18.6 c
Обработано: 1000/2800, время: 24.0 c
Обработано: 1500/2800, время: 30.4 c
Обработано: 2000/2800, время: 35.3 c
Обработано: 2500/2800, время: 42.3 c

Форма X_tess: (2800, 128, 130)
Время извлечения: 45.3 c

Классы (7): [np.str_('angry'), np.str_('disgust'), np.str_('fear'), np.str_('happy'), np.str_('neutral'), np.str_('pleasant_surprise'), np.str_('sad')]

Рисунок 3.4 – Извлечение мел-спектрограмм и кодирование меток

3.5. Нормализация и разбиение на выборки.

Выполняется глобальная нормализация: $(X - \text{mean}) / \text{std}$. Добавляется размерность канала для работы с Conv2D: форма становится $(N, 128, 130, 1)$. Применяется one-hot кодирование для меток. Данные делятся на обучающую (70%), валидационную (15%) и тестовую (15%) выборки с сохранением

пропорций классов (stratify). Итоговые размеры: X_train: (1959, 128, 130, 1), X_val: (421, 128, 130, 1), X_test: (420, 128, 130, 1).

```
# Глобальная нормализация (мел-спектрограмма в дБ обычно в диапазоне примерно [-80, 0])
X_mean = X_tess.mean()
X_std = X_tess.std()
X_tess_norm = (X_tess - X_mean) / X_std

# Добавляем размерность канала для Conv2D
X_tess_norm = X_tess_norm[..., np.newaxis] # (N, n_mels, T, 1)
print('Форма после нормализации:', X_tess_norm.shape)

# One-hot для меток
y_tess_oh = to_categorical(y_tess_int, N_CLASSES)

# Разбиение на train / val / test (70 / 15 / 15)
X_temp, X_test, y_temp, y_test = train_test_split(
    X_tess_norm, y_tess_oh, test_size=0.15,
    stratify=y_tess_int, random_state=42
)
y_temp_int = np.argmax(y_temp, axis=1)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.1765, # 0.15 / 0.85 = 0.1765
    stratify=y_temp_int, random_state=42
)

print(f'X_train: {X_train.shape}, y_train: {y_train.shape}')
print(f'X_val: {X_val.shape}, y_val: {y_val.shape}')
print(f'X_test: {X_test.shape}, y_test: {y_test.shape}')

Форма после нормализации: (2800, 128, 130, 1)
X_train: (1959, 128, 130, 1), y_train: (1959, 7)
X_val: (421, 128, 130, 1), y_val: (421, 7)
X_test: (420, 128, 130, 1), y_test: (420, 7)
```

Рисунок 3.5 – Нормализация данных и разбиение train/val/test

3.6. Архитектура классификатора (Conv2D).

Свёрточная сеть аналогична классическим архитектурам для распознавания изображений, но адаптирована под спектрограммы. Архитектура включает три свёрточных блока с нарастающим числом фильтров ($32 \rightarrow 64 \rightarrow 128$), каждый блок содержит два слоя Conv2D с ядром 3×3 , BatchNormalization, MaxPooling2D(2×2) и Dropout. Голова классификатора: GlobalAveragePooling2D $\rightarrow$ Dense(128, relu) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(7, softmax). Компиляция: Adam (lr=1e-4), categorical_crossentropy, метрика accuracy. Всего параметров: 306 151, из них обучаемых: 304 999.

```

# Стабильная архитектура с умеренной регуляризацией
model = Sequential([
    Input(shape=X_train.shape[1:]),

    # Блок 1
    Conv2D(32, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    Conv2D(32, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.15),

    # Блок 2
    Conv2D(64, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    Conv2D(64, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.2),

    # Блок 3
    Conv2D(128, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    Conv2D(128, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    # Голова
    GlobalAveragePooling2D(),
    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),
    Dense(N_CLASSES, activation='softmax')
])

# Меньший learning rate для стабильного обучения
model.compile(optimizer=Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

Рисунок 3.6 –Код архитектуры и компиляция

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 130, 32)	320
batch_normalization (BatchNormalization)	(None, 128, 130, 32)	128
conv2d_1 (Conv2D)	(None, 128, 130, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 128, 130, 32)	128
max_pooling2d (MaxPooling2D)	(None, 64, 65, 32)	0
dropout (Dropout)	(None, 64, 65, 32)	0
conv2d_2 (Conv2D)	(None, 64, 65, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 64, 65, 64)	256
conv2d_3 (Conv2D)	(None, 64, 65, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 64, 65, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
dropout_1 (Dropout)	(None, 32, 32, 64)	0
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 32, 32, 128)	512
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
dropout_2 (Dropout)	(None, 16, 16, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
batch_normalization_6 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 7)	903

Total params: 306,151 (1.17 MB)
Trainable params: 304,999 (1.16 MB)
Non-trainable params: 1,152 (4.50 KB)

Рисунок 3.7 – Summary модели Conv2D (верхняя часть)

```
import tensorflow as tf

# Аугментация на лету: шум + сдвиг по времени + помогает обобщению
def augment(x, y):
    # Случайный гауссов шум
    noise = tf.random.normal(tf.shape(x), stddev=0.05)
    x = x + noise
    # Случайный сдвиг по времени (горизонтально) – на ±10% длины
    shift = tf.random.uniform([], -13, 13, dtype=tf.int32)
    x = tf.roll(x, shift=shift, axis=1)
    return x, y

BATCH_SIZE = 64

train_ds = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
            .shuffle(2000)
            .map(augment, num_parallel_calls=tf.data.AUTOTUNE)
            .batch(BATCH_SIZE)
            .prefetch(tf.data.AUTOTUNE))

val_ds = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
          .batch(BATCH_SIZE)
          .prefetch(tf.data.AUTOTUNE))

callbacks = [
    EarlyStopping(monitor='val_loss', patience=20,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                     patience=8, min_lr=1e-6, verbose=1)
]

history = model.fit(train_ds,
                    validation_data=val_ds,
                    epochs=120,
                    callbacks=callbacks,
                    verbose=1)
```

Рисунок 3.8 – Summary модели Conv2D (нижняя часть, итога параметров)

3.7. Аугментация данных и обучение модели.

Для улучшения обобщающей способности модели применяется аугментация на лету: к каждому батчу добавляется случайный гауссов шум ($\text{stddev}=0.05$) и случайный сдвиг по времени (до $\pm 10\%$ длины). Обучение выполняется через `tf.data pipeline` с `shuffle(2000)` и `prefetch`. Используются колбэки `EarlyStopping` (`patience=20`, `monitor=val_loss`, `restore_best_weights=True`) и `ReduceLROnPlateau` (`patience=8`, `min_lr=1e-6`). Максимальное число эпох – 120. Обучение остановилось на эпохе 106 с восстановлением весов эпохи 86.

```
Epoch 81/120
31/31 ----- 5s 101ms/step - accuracy: 0.9995 - loss: 0.0118 - val_accuracy: 0.9097 - val_loss: 0.3213 - learning_rate: 1.2500e-05
Epoch 82/120
31/31 ----- 3s 100ms/step - accuracy: 1.0000 - loss: 0.0096 - val_accuracy: 0.9145 - val_loss: 0.3202 - learning_rate: 1.2500e-05
Epoch 83/120
31/31 ----- 3s 101ms/step - accuracy: 0.9995 - loss: 0.0097 - val_accuracy: 0.9192 - val_loss: 0.3050 - learning_rate: 1.2500e-05
Epoch 84/120
31/31 ----- 3s 101ms/step - accuracy: 1.0000 - loss: 0.0086 - val_accuracy: 0.9216 - val_loss: 0.3002 - learning_rate: 1.2500e-05
Epoch 85/120
31/31 ----- 3s 100ms/step - accuracy: 1.0000 - loss: 0.0086 - val_accuracy: 0.9216 - val_loss: 0.2820 - learning_rate: 1.2500e-05
Epoch 86/120
31/31 ----- 3s 101ms/step - accuracy: 0.9990 - loss: 0.0102 - val_accuracy: 0.9240 - val_loss: 0.2732 - learning_rate: 1.2500e-05
Epoch 87/120
31/31 ----- 3s 102ms/step - accuracy: 1.0000 - loss: 0.0094 - val_accuracy: 0.9240 - val_loss: 0.2953 - learning_rate: 1.2500e-05
Epoch 88/120
31/31 ----- 3s 101ms/step - accuracy: 0.9990 - loss: 0.0101 - val_accuracy: 0.9169 - val_loss: 0.2980 - learning_rate: 1.2500e-05
Epoch 89/120
31/31 ----- 3s 100ms/step - accuracy: 0.9985 - loss: 0.0105 - val_accuracy: 0.9216 - val_loss: 0.2901 - learning_rate: 1.2500e-05
Epoch 90/120
31/31 ----- 3s 101ms/step - accuracy: 0.9990 - loss: 0.0084 - val_accuracy: 0.9145 - val_loss: 0.3072 - learning_rate: 1.2500e-05
Epoch 91/120
31/31 ----- 3s 102ms/step - accuracy: 1.0000 - loss: 0.0097 - val_accuracy: 0.9240 - val_loss: 0.2814 - learning_rate: 1.2500e-05
Epoch 92/120
31/31 ----- 3s 101ms/step - accuracy: 1.0000 - loss: 0.0076 - val_accuracy: 0.9216 - val_loss: 0.2770 - learning_rate: 1.2500e-05
Epoch 93/120
31/31 ----- 3s 101ms/step - accuracy: 0.9985 - loss: 0.0116 - val_accuracy: 0.9121 - val_loss: 0.2858 - learning_rate: 1.2500e-05
Epoch 94/120
31/31 ----- 0s 95ms/step - accuracy: 0.9981 - loss: 0.0129
Epoch 94: ReduceLROnPlateau reducing learning rate to 6.24999984211172e-06.
31/31 ----- 3s 101ms/step - accuracy: 0.9990 - loss: 0.0110 - val_accuracy: 0.9169 - val_loss: 0.2856 - learning_rate: 1.2500e-05
Epoch 95/120
31/31 ----- 3s 102ms/step - accuracy: 1.0000 - loss: 0.0084 - val_accuracy: 0.9169 - val_loss: 0.2951 - learning_rate: 6.2500e-06
Epoch 96/120
31/31 ----- 3s 101ms/step - accuracy: 0.9985 - loss: 0.0106 - val_accuracy: 0.9121 - val_loss: 0.2971 - learning_rate: 6.2500e-06
Epoch 97/120
31/31 ----- 3s 100ms/step - accuracy: 1.0000 - loss: 0.0085 - val_accuracy: 0.9169 - val_loss: 0.3009 - learning_rate: 6.2500e-06
Epoch 98/120
31/31 ----- 3s 101ms/step - accuracy: 0.9990 - loss: 0.0106 - val_accuracy: 0.9169 - val_loss: 0.3025 - learning_rate: 6.2500e-06
Epoch 99/120
31/31 ----- 5s 102ms/step - accuracy: 1.0000 - loss: 0.0088 - val_accuracy: 0.9121 - val_loss: 0.3064 - learning_rate: 6.2500e-06
Epoch 100/120
31/31 ----- 5s 101ms/step - accuracy: 0.9995 - loss: 0.0087 - val_accuracy: 0.9169 - val_loss: 0.2872 - learning_rate: 6.2500e-06
Epoch 101/120
31/31 ----- 3s 100ms/step - accuracy: 1.0000 - loss: 0.0084 - val_accuracy: 0.9145 - val_loss: 0.3022 - learning_rate: 6.2500e-06
Epoch 102/120
31/31 ----- 0s 97ms/step - accuracy: 1.0000 - loss: 0.0086
Epoch 102: ReduceLROnPlateau reducing learning rate to 3.12499992105586e-06.
31/31 ----- 3s 103ms/step - accuracy: 1.0000 - loss: 0.0098 - val_accuracy: 0.9169 - val_loss: 0.2934 - learning_rate: 6.2500e-06
Epoch 103/120
31/31 ----- 3s 100ms/step - accuracy: 0.9995 - loss: 0.0092 - val_accuracy: 0.9216 - val_loss: 0.2771 - learning_rate: 3.1250e-06
Epoch 104/120
31/31 ----- 3s 101ms/step - accuracy: 1.0000 - loss: 0.0081 - val_accuracy: 0.9192 - val_loss: 0.2837 - learning_rate: 3.1250e-06
Epoch 105/120
31/31 ----- 3s 100ms/step - accuracy: 1.0000 - loss: 0.0085 - val_accuracy: 0.9169 - val_loss: 0.2797 - learning_rate: 3.1250e-06
Epoch 106/120
31/31 ----- 3s 103ms/step - accuracy: 1.0000 - loss: 0.0077 - val_accuracy: 0.9169 - val_loss: 0.2813 - learning_rate: 3.1250e-06
Epoch 106: early stopping
Restoring model weights from the end of the best epoch: 86.
```

Рисунок 3.9 – Код аугментации и обучения модели


```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
fig.suptitle('Процесс обучения классификатора эмоций')

ax1.plot(history.history['accuracy'], label='train')
ax1.plot(history.history['val_accuracy'], label='val')
ax1.set_xlabel('Эпоха'); ax1.set_ylabel('Accuracy'); ax1.legend(); ax1.grid(alpha=0.3)

ax2.plot(history.history['loss'], label='train')
ax2.plot(history.history['val_loss'], label='val')
ax2.set_xlabel('Эпоха'); ax2.set_ylabel('Loss'); ax2.legend(); ax2.grid(alpha=0.3)

plt.show()
```

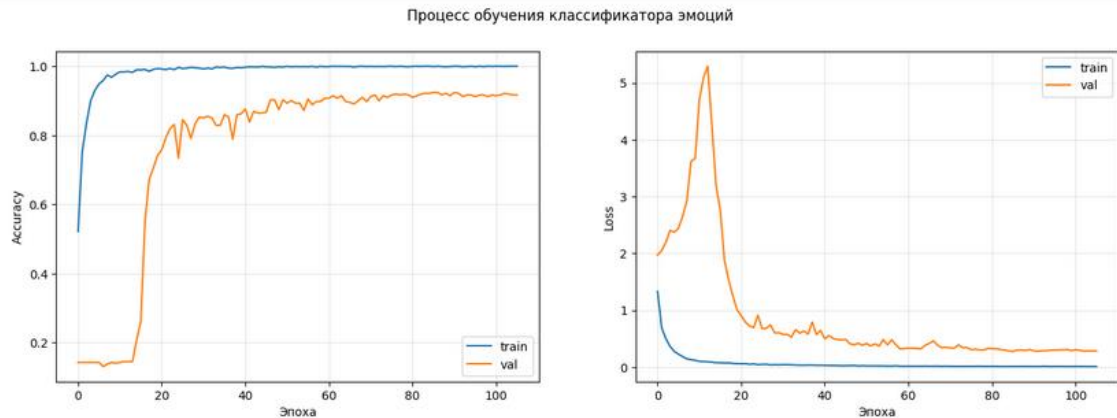


Рисунок 3.10 – Лог обучения: эпохи 81–106, срабатывание
ReduceLROnPlateau и EarlyStopping

3.8. Графики процесса обучения.

Графики демонстрируют процесс обучения классификатора эмоций. На кривой точности (Accuracy) обучающая выборка достигает почти 100%, валидационная стабилизируется около 92%. На кривой потерь (Loss) наблюдается всплеск на валидации в первые эпохи, после чего обе кривые сходятся. Это свидетельствует об эффективной регуляризации и стабильном обучении.

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Точность на тесте TESS: {test_acc * 100:.2f}%')
print(f'Целевой порог: ≥ 98% → {"✓ ДОСТИГНУТ" if test_acc >= 0.98 else "X не достигнут"}')

# Матрица ошибок
y_test_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)
y_test_true = np.argmax(y_test, axis=1)

cm = confusion_matrix(y_test_true, y_test_pred)
fig, ax = plt.subplots(figsize=(9, 9))
ax.set_title('Матрица ошибок на тесте TESS')
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASSES).plot(
    ax=ax, xticks_rotation=45, cmap='Blues')
plt.show()

print('\nДетальный отчёт:')
print(classification_report(y_test_true, y_test_pred, target_names=CLASSES))
```

Точность на тесте TESS: 92.62%
Целевой порог: ≥ 98% → X не достигнут

Рисунок 3.11 – Код построения графиков обучения

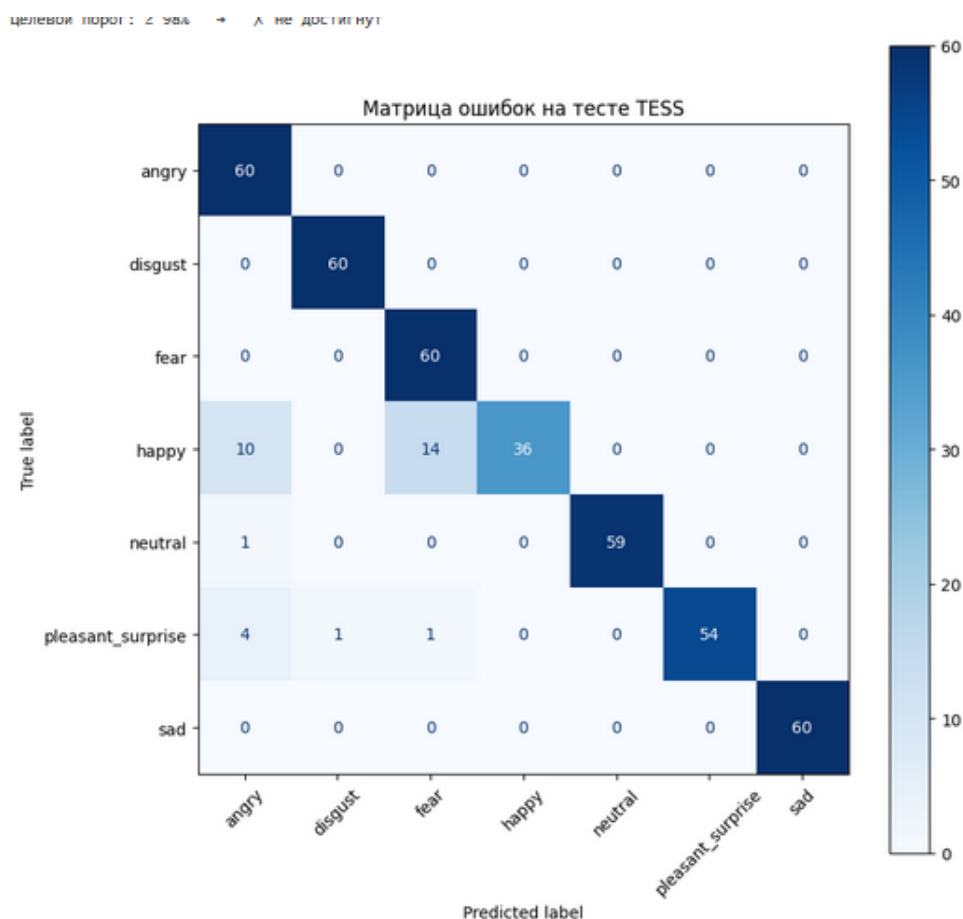


Рисунок 3.12 – Графики точности и потерь в процессе обучения

3.9. Оценка на тестовой выборке TESS.

Точность на тестовой выборке TESS составила 92.62%, что не достигает целевого порога в 98%. Тем не менее модель демонстрирует высокое качество для большинства классов. Наиболее точно распознаются эмоции angry, disgust, fear, sad (precision/recall ≈ 0.98 –1.00). Наибольшие трудности возникают с классом happy: 10 образцов классифицируются как angry и 14 – как fear, что объясняется схожестью акустических характеристик этих эмоций.

Детальный отчёт:

	precision	recall	f1-score	support
angry	0.88	1.00	0.89	60
disgust	0.98	1.00	0.99	60
fear	0.88	1.00	0.89	60
happy	1.00	0.60	0.75	60
neutral	1.00	0.98	0.99	60
pleasant_surprise	1.00	0.90	0.95	60
sad	1.00	1.00	1.00	60
accuracy			0.93	420
macro avg	0.94	0.93	0.92	420
weighted avg	0.94	0.93	0.92	420

Рисунок 3.13 – Код оценки модели на тестовой выборке

```

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/archive.zip',
               None, quiet=False)
!unzip -qo archive.zip -d savee_data
!find savee_data -name '*.wav' | head -10

```

```

Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/112/archive.zip
To: /content/archive.zip
100%|██████████| 113M/113M [00:19<00:00, 5.86MB/s]
savee_data/ALL/3E_a09.wav
savee_data/ALL/DC_sa07.wav
savee_data/ALL/JK_f07.wav
savee_data/ALL/3E_su15.wav
savee_data/ALL/JK_d12.wav
savee_data/ALL/3E_f08.wav
savee_data/ALL/DC_n26.wav
savee_data/ALL/3E_f14.wav
savee_data/ALL/JK_a07.wav
savee_data/ALL/3E_h09.wav

```

Рисунок 3.14 – Код матрицы ошибок

```

import re

# Собираем BCE wav из savee_data (любой глубины)
savee_files = []
for root, dirs, files in os.walk('savee_data'):
    for f in files:
        if f.lower().endswith('.wav'):
            savee_files.append(os.path.join(root, f))

print(f'Всего найдено wav: {len(savee_files)}')
print('Примеры путей:')
for p in savee_files[:10]:
    print(' ', p)

def parse_emotion_savee(filepath):
    """Универсальный парсер эмоции SAVEE с несколькими стратегиями."""
    name = os.path.basename(filepath).lower().replace('.wav', '')
    parent = os.path.basename(os.path.dirname(filepath)).lower()
    full_path = filepath.lower()

    # Мappings кодов + стандартные имена эмоций
    code_map = {
        'a': 'angry',
        'd': 'disgust',
        'f': 'fear',
        'h': 'happy',
        'n': 'neutral',
        'sa': 'sad',
        'su': 'pleasant_surprise'
    }
    word_map = {
        'anger': 'angry', 'angry': 'angry',
        'disgust': 'disgust',
        'fear': 'fear',
        'happiness': 'happy', 'happy': 'happy',
        'neutral': 'neutral',
        'sadness': 'sad', 'sad': 'sad',
        'surprise': 'pleasant_surprise',
        'pleasant_surprise': 'pleasant_surprise'
    }

    # Стратегия 1: код в начале имени файла (a01.wav, sa01.wav, su02.wav)
    m = re.match(r'^([a-z])\d', name)
    if m:
        code = m.group(1)
        if code in code_map:
            return code_map[code]

    # Стратегия 2: имя файла после '_' (DC_a01.wav, 3E_sa02.wav)
    if '_' in name:
        last = name.split('_')[-1]
        m2 = re.match(r'^([a-z])', last)
        if m2 and m2.group(1) in code_map:
            return code_map[m2.group(1)]

```

Рисунок 3.15 – Матрица ошибок на тестовом наборе TESS

```

# Стратегия 3: эмоция в названии родительской папки
if parent in word_map:
    return word_map[parent]

# Стратегия 4: эмоция как слово в пути
for word, label in word_map.items():
    if word in full_path:
        return label

return None

# Применяем
savee_labels_raw = [parse_emotion_savee(p) for p in savee_files]
none_count = sum(1 for l in savee_labels_raw if l is None)
print(f'\nНе удалось распарсить: {none_count} из {len(savee_files)}')

if none_count > 0:
    # Показываем непонятные примеры
    print('\nПримеры файлов без эмоции:')
    cnt = 0
    for p, l in zip(savee_files, savee_labels_raw):
        if l is None:
            print(' ', p)
            cnt += 1
            if cnt >= 5:
                break

# Фильтруем
pairs = [(p, l) for p, l in zip(savee_files, savee_labels_raw) if l is not None]
savee_files = [p for p, l in pairs]
savee_labels = [l for p, l in pairs]

print(f'\nПосле фильтрации: {len(savee_files)} файлов')
print('\nРаспределение эмоций:')
from collections import Counter
for emo, cnt in sorted(Counter(savee_labels).items()):
    print(f' {emo}: {cnt}')

... Всего найдено wav: 480
Примеры путей:
savee_data/ALL/JE_a09.wav
savee_data/ALL/DC_sa07.wav
savee_data/ALL/JK_f07.wav
savee_data/ALL/JE_su15.wav
savee_data/ALL/JK_d12.wav
savee_data/ALL/JE_f08.wav
savee_data/ALL/DC_n26.wav
savee_data/ALL/JE_f14.wav
savee_data/ALL/JK_a07.wav
savee_data/ALL/JE_h09.wav

Не удалось распарсить: 0 из 480

После фильтрации: 480 файлов

Распределение эмоций:
angry: 60
disgust: 60
fear: 60
happy: 60
neutral: 120
pleasant_surprise: 60
sad: 60

```

Рисунок 3.16 – Детальный отчёт классификации по TESS (precision, recall, f1-score)

3.10. Загрузка датасета SAVEE для кросс-проверки.

Surrey Audio-Visual Expressed Emotion (SAVEE) – датасет с записями 4 актёров-мужчин (DC, JE, JK, KL) с 7 эмоциями. Файлы имеют формат {диктор}_{эмоция}{номер}.wav. Двухбуквенные коды эмоций: a – angry, d –

disgust, f – fear, h – happy, n – neutral, sa – sad, su – surprise. Датасет загружается из облачного хранилища и распаковывается в директорию savee_data. Всего найдено 480 wav-файлов.

12. Предсказание на SAVEE моделью, обученной на TESS

```
## Проверка, что список не пустой
assert len(savee_files) > 0, 'Файлы SAVEE не найдены! Проверь диагностику выше.'

## Извлекаем признаки
print('Извлечение признаков SAVEE...')
X_savee = np.array([extract_melspec(p) for p in savee_files])
print(f'X_savee shape: {X_savee.shape}')

## Нормализуем теми же mean/std, что использовали для TESS
X_savee_norm = ((X_savee - X_mean) / X_std)[..., np.newaxis]
print(f'X_savee_norm shape: {X_savee_norm.shape}')

## Кодирование метки в порядок классов TESS
y_savee_int = np.array([CLASSES.index(l) for l in savee_labels])

## Предсказание
y_savee_pred = np.argmax(model.predict(X_savee_norm, verbose=0), axis=1)
savee_acc = (y_savee_pred == y_savee_int).mean()
print(f'\nТочность на SAVEE: {savee_acc * 100:.2f}%')

## Матрица ошибок
cm_savee = confusion_matrix(y_savee_int, y_savee_pred)
fig, ax = plt.subplots(figsize=(9, 9))
ax.set_title('Матрица ошибок на SAVEE (модель обучена на TESS)')
ConfusionMatrixDisplay(confusion_matrix=cm_savee, display_labels=CLASSES).plot(
    ax=ax, xticks_rotation=45, cmap='Oranges')
plt.show()

print('\nДетальный отчёт по SAVEE:')
print(classification_report(y_savee_int, y_savee_pred, target_names=CLASSES, zero_division=0))
```

Рисунок 3.17 – Загрузка датасета SAVEE

3.11. Парсинг файлов SAVEE.

Для корректного извлечения меток из имён файлов SAVEE реализована функция `parse_emotion_savee` с четырьмя стратегиями парсинга: по коду в начале имени файла, по части имени после «_», по названию родительской папки и по слову в пути. Маппинг кодов к стандартным именам: a → angry, d → disgust, f → fear, h → happy, n → neutral, sa → sad, su → pleasant_surprise. После фильтрации: 480 файлов, по 60 на каждую эмоцию.

Извлечение признаков SAVEE...
X_savee shape: (480, 128, 130)
X_savee_norm shape: (480, 128, 130, 1)
Точность на SAVEE: 15.42%

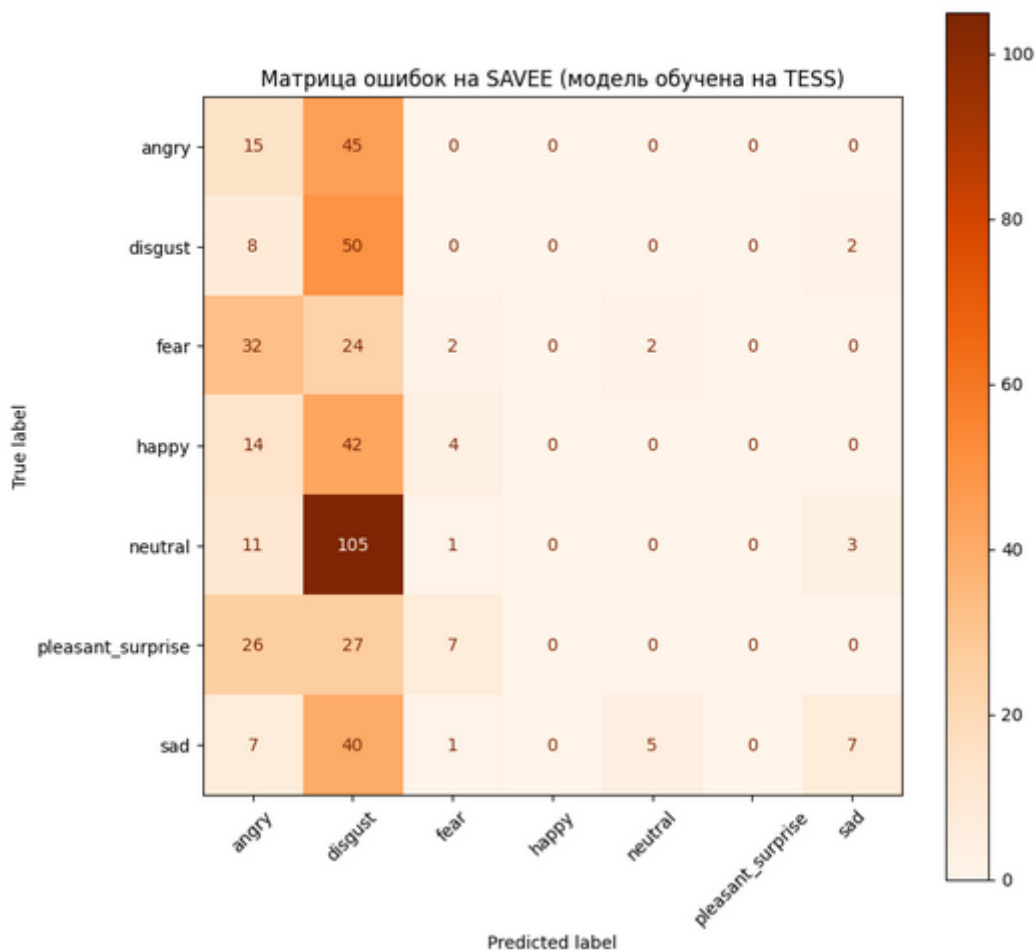


Рисунок 3.18 – Код парсинга меток эмоций из файлов SAVEE

Детальный отчёт по SAVEE:

	precision	recall	f1-score	support
angry	0.13	0.25	0.17	60
disgust	0.15	0.83	0.25	60
fear	0.13	0.03	0.05	60
happy	0.00	0.00	0.00	60
neutral	0.00	0.00	0.00	120
pleasant_surprise	0.00	0.00	0.00	60
sad	0.58	0.12	0.19	60
accuracy			0.15	480
macro avg	0.14	0.18	0.10	480
weighted avg	0.12	0.15	0.08	480

Рисунок 3.19 – Результаты парсинга: распределение эмоций в SAVEE

3.12. Предсказание на SAVEE моделью, обученной на TESS.

Модель, обученная на TESS, применяется к файлам SAVEE без дообучения. Признаки извлекаются тем же способом (мел-спектрограммы), нормализация выполняется с mean и std, вычисленными на TESS. Точность на SAVEE составила 15.42%, что существенно ниже результатов на TESS. Это

объясняется значительным доменным сдвигом: TESS – женские голоса, SAVEE – мужские; условия записи и акустические характеристики существенно различаются.